Clustering Techniques: A Literature Survey of Recent Algorithms and Techniques

Scope and sourcing. As for this survey, sources are limited to published journals, conference proceedings and book series; preprint-server versions are not used as references even where they exist. The review of each recent paper (Section 3) is based on its published abstract and bibliographic record as retrieved on 15 October 2026, not on a full-text reading; where Section 4 describes the mechanics of an algorithm in more detail, that detail comes from the foundational papers (density peaks, landmark spectral clustering, HDBSCAN*) and is stated as such. Reported performance figures are the authors' own and have not been independently verified unless Section 7.

What is and is not implemented. Four algorithm families are implemented and tested: density peaks (with a kNN-density variant), landmark-based spectral clustering, HDBSCAN* (from scratch, cross-checked against scikit-learn) and centroid-based deep clustering with a BRB-style soft reset. Torque Clustering is reviewed but deliberately not implemented: the authors' code is obfuscated MATLAB under a non-commercial licence, and a clean-room attempt from the published description did not reproduce the behaviour the paper reports, so no misleading approximation is shipped (Section 4.5).

2. Background and taxonomy

Clustering partitions unlabeled data so that members of a group are more similar to each other than to members of other groups. Methods differ in what they assume about cluster shape, whether the number of clusters must be supplied, and how they scale.

Family	Representative methods	Number of clusters	Typical weakness
Partitional	k-means and variants	Given	Convex, similar-size clusters
Hierarchical	Agglomerative / single linkage, HDBSCAN*	Cut level or stability	$O(n^2)$ memory in the basic form
Density-based	DBSCAN, density peaks (DPC), HDBSCAN*	Automatic	Sensitive to density scale / cut-off parameters
Spectral / graph	Spectral clustering, landmark and anchor-graph variants	Given	$O(n^3)$ unless approximated
Deep	Autoencoder, contrastive and prototype-based networks	Usually given	Training instability, dependence on augmentations
Multi-view / ensemble	Anchor-graph multi-view, consensus clustering	Given	View weighting, fusion cost

Table 1. Families of clustering methods considered in this survey.

Evaluation. With ground-truth labels we use the Adjusted Rand Index (ARI), Normalized Mutual Information (NMI) and Hungarian-matched clustering accuracy (ACC). For methods that output noise (HDBSCAN*), noise points are treated as one additional label when computing these scores, which penalises methods that discard many points.

3. Review of the literature

3.1 Surveys

Ren et al. [1] (IEEE TNNLS, 2024) survey deep clustering, and Zhou et al. [2] (ACM Computing Surveys 57(3), 2025) give a taxonomy and discuss open challenges and future directions. Both are the natural entry points for the deep-clustering literature, and the second also organises methods by learning paradigm (for example, contrastive deep clustering, where positive pairs come from data augmentation).

3.2 Parameter-free clustering: Torque Clustering

Yang and Lin [3] (IEEE TPAMI 47(7), July 2025) propose Torque Clustering, based on the principle that a cluster should merge with its nearest neighbour of higher mass, unless both clusters have relatively large masses and the distance between them is also large. Peaks in mass and distance identify incorrect mergers, which are removed. The authors present it as a competitive alternative to existing methods across synthetic and real data sets, and the university press release describes it as parameter-free, able to determine the number of clusters and identify noise. The inspiration is the torque balance in gravitational galaxy mergers. A press-release figure of 97.7% average AMI over 1,000 data sets has circulated; it is the authors' claim and is not verified here. The official code is MATLAB (a MEX binary and obfuscated `.p` files) released under CC BY-NC-SA 4.0, i.e. non-commercial; the authors list an unofficial community Python port and state that it may differ from the paper.

3.3 Density-based and hierarchical density methods

Density peaks clustering (DPC) [4] (Science, 2014) finds centres as points that are both locally dense and far from any denser point, with no iteration. Its sensitivity to the cut-off distance and to error propagation in label assignment has motivated many variants. Che et al. [5] (IEEE Access 2022) replace the cut-off density by a potential-and-centrality measure and propagate labels from centres by a diffusion-strength mechanism. Zhou et al. [6] (IEEE Access 12, 2024) redefine density with k-nearest neighbours and shared nearest neighbours and add a multi-cluster fusion step, targeting data with large density differences and cascading assignment errors.

HDBSCAN (HDBSCAN-star) [7, 8] (PAKDD 2013; ACM TKDD 2015) builds a hierarchy of DBSCAN* solutions over all density levels and extracts the most stable clusters, so only a minimum cluster size is required. McInnes and Healy [9] (IEEE ICDMW 2017) give an accelerated implementation that is the basis of the widely used library. It remains the standard baseline for varying-density data.

3.4 Scalable spectral and anchor-graph clustering

Spectral clustering handles non-convex clusters but costs $O(n^3)$. Chen and Cai [10] (AAAI 2011; extended in [11], IEEE Trans. Cybernetics) select $p \ll n$ landmarks, represent every point as a combination of its nearest landmarks and compute the embedding from the small landmark representation, giving cost linear in n. Wang et al. [12] (ICASSP 2020) argue that the second-order transition view of anchor-graph spectral clustering distorts boundary samples and propose a random-walk-Laplacian variant. Yang et al. [13] (IEEE TKDE 2025) extend anchor graphs to multiple views with a min-max view-combination model and show the solution is globally optimal. An ICML 2026 poster, SC-FAGC [14], adds explicit cluster-size constraints to anchor-based spectral clustering via an optimal-transport solver, targeting degenerate unbalanced partitions (listed from the conference page only).

3.5 Deep clustering

Sample stability. Cheng et al. [15] (IJCAI 2024) propose DECS, which removes hand-built pseudo targets and instead pulls each sample toward its centroid with a stability term based on its relationship to all centroids. Li et al. [19] (ICML 2025) show that per-sample stability of the learned representation during training correlates with clustering accuracy and memorisation, and turn this into supervision signals that can be added to existing deep clustering methods.

Online and centerless clustering. Yan et al. [16] (ECCV 2024) introduce Probability Aggregation Clustering, a centerless fuzzy formulation, and an online mini-batch module that avoids the alternation between feature clustering and model training; the deep version (DPAC) combines it with a contrastive loss.

Training dynamics. Miklautz et al. [18] (ICLR 2025) observe that centroid-based deep clustering saturates early, that periodic reclustering does not remove the plateau (the "reclustering barrier"), and propose BRB, which combines reclustering with a soft reset of the encoder; combined with a contrastive loss it is competitive with state-of-the-art methods.

Other directions. Kumar et al. [17] (TMLR 2024) build hard negative pairs through Hamiltonian Monte Carlo sampling so that mode-seeking clustering can be applied without specifying k. Ronen et al. [20] (CVPR 2022) cluster with an unknown number of clusters inside the network.

3.6 Comparison

Ref.	Method (venue, year)	Core idea	Reported strength / limitation
[3]	Torque Clustering (TPAMI 2025)	Mass–distance merging; peaks mark wrong mergers	Parameter-free, noise detection / code non-commercial, not independently checked
[5]	DPC-PMDS (IEEE Access 2022)	Potential density + diffusion label spread	Complex shapes, uneven density / extra computation
[6]	SM-DPC (IEEE Access 2024)	SNN density + multi-cluster fusion	Arbitrary density / more design choices
[7–9]	HDBSCAN* (PAKDD 2013 … ICDMW 2017)	Density hierarchy + stability extraction	Varying density, noise / $O(n^2)$ basic cost
[10, 11]	LSC (AAAI 2011, TCyb 2015)	Landmark sparse representation	Linear in n / landmark choice
[13]	SMMSC (TKDE 2025)	Min-max multi-view anchor graphs	Large multi-view data / needs k
[15]	DECS (IJCAI 2024)	Sample stability, no pseudo targets	Simple objective / evaluated on five data sets
[16]	DPAC (ECCV 2024)	Centerless online probability aggregation	Stable online training / vision-focused
[18]	BRB (ICLR 2025)	Soft reset + reclustering	Breaks early plateau / needs reclustering schedule
[19]	LFSS (ICML 2025)	Sample-stability supervision	Plug-in for existing methods / extra training signal

Table 2. Summary of reviewed papers (strengths and limitations as stated in the abstracts or evident from the design).

4. Algorithms: working principles and methodology

4.1 Density Peaks Clustering (implemented)

Principle. Cluster centres are surrounded by lower-density points and lie far from any point of higher density. For every point i the algorithm computes a local density ρ_i and the distance δ_i to the nearest point of higher density (for the global density maximum, the largest distance to any point).

$$\rho_i = \Sigma_j \exp(-(d_{ij} / d_c)^2) \qquad \delta_i = \min_{j \text{ with } \rho_j > \rho_i} d_{ij}$$

Procedure. (1) Compute pairwise distances and set d_c to a low percentile (about 2%) of all distances. (2) Compute ρ and δ. (3) Select centres as points with large $\gamma = \tilde{\rho} \cdot \tilde{\delta}$ (normalised product), either the top k or automatically by an elbow rule: the largest ratio gap among the top-γ points. (4) Visit remaining points in decreasing ρ and give each the label of its nearest higher-density neighbour. Time and memory are $O(n^2)$.

Implementation notes. DensityPeaks supports cut-off, Gaussian and kNN density. The kNN variant is a simplified relative of the improvements in [5, 6], not a re-implementation of either. **Known weakness (seen in Section 7):** a wrongly assigned point propagates its label to everything that follows it in the density order, and the method cannot separate concentric rings because the nearest higher-density neighbour is on the same ring only by luck.

4.2 Landmark-based Spectral Clustering (implemented)

Principle. Replace the n×n affinity matrix by a sparse n×p matrix Z between points and p landmarks, and obtain the spectral embedding from the SVD of a normalised Z, never forming the n×n graph.

Procedure. (1) Choose p landmarks with k-means. (2) For each point keep its r nearest landmarks and set z_{ij} proportional to a Gaussian kernel of the distance, normalised so each row sums to 1. (3) Degree $d_i = z_i \cdot (Z^T 1)$; $\hat{Z} = D^{\wedge}(-1/2)\, Z$. (4) The k leading left singular vectors of $\hat{Z}$ equal the leading eigenvectors of the implied normalised affinity $\hat{Z}\,\hat{Z}^T$. (5) Row-normalise this embedding and run k-means. Cost is O(n p d) for the graph and O(n p k) for the SVD, i.e. linear in n. Parameters are p, r and k. The multi-view method [13] and the random-walk variant [12] modify the graph and the weighting but keep this anchor-graph skeleton.

4.3 HDBSCAN* (implemented from scratch)

Principle. Run DBSCAN* at every density level simultaneously and keep the clusters that persist longest.

Procedure. (1) Core distance of a point = distance to its m-th nearest neighbour (the point counts as its own first neighbour, as in scikit-learn). (2) Mutual reachability d*mr(i, j) = max(core(i), core(j), d(i, j)). (3) Minimum spanning tree of the d*mr graph; sorting its edges and merging gives the single-linkage dendrogram. (4) Condense the dendrogram: a split is real only if both children have at least min*cluster*size points; otherwise the smaller child's points "fall off" as the parent continues. With λ = 1/distance, (5) the stability of a cluster is the sum over its points of ($\lambda_{leave} - \lambda_{birth}$). (6) Excess-of-mass selection, bottom-up: keep a cluster if its stability is at least the sum of its selected descendants' stabilities. Points in no selected cluster are noise. The implementation is dense $O(n^2)$; fast variants for low dimension reach near O(n log n) [9]. In a development check on the six 2-D sets the labels agreed with scikit-learn's HDBSCAN (ARI between the two ≥ 0.996, exactly 1.000 on four); tests.py repeats this check on the moons set.

4.4 Centroid-based deep clustering with reclustering and a BRB-style soft reset (implemented at toy scale)

Principle. An autoencoder learns an embedding z; clustering is made explicit by pulling each embedding toward its assigned centroid. Over training the embedding compresses around the initial centroids and gains plateau (the reclustering barrier [18]).

Procedure. (1) Pre-train an MLP autoencoder with reconstruction loss. (2) Initialise centroids by k-means on z. (3) Minimise $L = MSE(x, \hat{x}) + \beta \cdot mean((z - c_a)^2)$, *a = nearest centroid; update centroids to the cluster means each epoch. (4) Variants: no reclustering; every T epochs re-run k-means on z; or (BRB-style) before each reclustering apply shrink-and-perturb to the encoder,* $\theta \leftarrow \lambda\theta + (1-\lambda)\theta_{fresh}$, with θ_{fresh} a new random initialisation ($\lambda = 0.8$ here). Back-propagation and Adam are hand-written in NumPy (PyTorch could not be installed in the build environment), which also keeps the code dependency-light.

Caveat. This is a compact reading of the published idea, not the authors' code: backbones, hyper-parameters and reset details of [18] differ, so the numbers in Section 7 neither confirm nor refute that paper. DECS [15], DPAC [16] and LFSS [19] are described in Section 3 but not implemented.

4.5 Torque Clustering (reviewed, not implemented)

From the published description [3]: points start as clusters of mass one; each cluster is drawn toward a neighbour of higher mass; merges that pair two relatively large masses at a relatively large distance (peaks in mass and distance) are treated as errors and cut. The number of clusters then follows from the cut set. The abstract and public descriptions do not fully specify the distance definition between clusters or the exact peak thresholds, and the reference code is obfuscated, so these details cannot be checked.

What was tried. A clean-room implementation of the nearest-higher-mass merge tree with several readings of the peak rule (cut by torque ranking; per-round mean thresholds; mean-plus-k·std thresholds; single-link versus cluster-centre distance) was tested on the six 2-D sets. None gave stable, sensible results (for example 10 to 65 clusters for three-blob data depending on the threshold, and ARI below 0.6 in the best settings), so it is not included. This says nothing about the original method; it shows that the description alone is insufficient to reproduce it. Readers who need Torque Clustering should use the authors' repository directly and confirm that its licence permits their use.

5. Code package

File	Content
clustering/dpc.py	DensityPeaks: cut-off / Gaussian / kNN density, automatic or given k
clustering/lsc.py	LandmarkSpectral: k-means landmarks, sparse Z, SVD embedding
clustering/hdbscan_star.py	HDBSCANStar: mutual reachability, MST, condensed tree, EOM selection
clustering/deep_brb.py	DeepCentroidClustering: NumPy autoencoder, modes none / recluster / brb
clustering/datasets.py, metrics.py	Six 2-D benchmarks, digits; ARI, NMI, ACC
run_demo.py	Runs all experiments, writes tables and figures to results/
tests.py	Four correctness checks, including agreement with scikit-learn HDBSCAN

File	Content
README.md, requirements.txt	Instructions and dependencies

Table 3. Contents of clustering_survey_code/.

6. Execution instructions

Requirements: Python 3.10 or newer; NumPy, SciPy, scikit-learn, matplotlib and pandas. No GPU and no downloads are needed. Developed and run with Python 3.13, NumPy 2.5 and scikit-learn 1.9.

```
cd clustering_survey_code
python -m venv .venv && source .venv/bin/activate   # optional
pip install -r requirements.txt
python tests.py                # 4 checks, about 10 s
python run_demo.py             # all experiments, about 1-2 min
python run_demo.py --only 2d     # classical methods only
python run_demo.py --only deep   # deep-clustering experiment only
```

Minimal use in your own code:

```
from clustering import DensityPeaks, LandmarkSpectral, HDBSCANStar, DeepCentroidClustering

labels = HDBSCANStar(min_cluster_size=15).fit_predict(X)       # -1 = noise
labels = LandmarkSpectral(n_clusters=4, n_landmarks=200).fit_predict(X)
labels = DensityPeaks().fit_predict(X)                         # automatic centres
labels = DeepCentroidClustering(10, mode='brb').fit_predict(X)  # X scaled to [0, 1]
```

Outputs are written to results/: results_2d.csv, results_2d_ari_table.csv, fig_2d.png, fig_dpc_decision_graph.png, results_deep_runs.csv, results_deep_summary.csv and fig_deep_curve.png. The implementations are dense or $O(n^2)$ in memory except landmark spectral clustering, so they are intended for up to a few thousand points.

7. Experimental results

All numbers below were produced by `python run_demo.py` on the six synthetic 2-D data sets (about 900 points each, standardised) and on scikit-learn's 8×8 digits (1,797 × 64). Synthetic data and one fixed seed per data set are small evidence; they illustrate behaviour and are not a benchmark.

7.1 Classical algorithms on 2-D data

Data set	k-means (k given)	DPC (auto)	DPC (k given)	LSC (k given)	HDBSCAN* (auto)
blobs	0.86	0.83	0.83	0.84	0.37
moons	0.49	0.74	0.74	1.00	1.00
circles	0.00	0.00	0.00	1.00	1.00
aniso	0.59	0.97	0.97	0.97	0.91
varied_density	0.98	0.00	0.99	1.00	0.98
arcs_blob	0.68	0.67	0.67	1.00	0.81

Table 4. Adjusted Rand Index (higher is better). “auto” = number of clusters found by the method; “k given” = true k supplied.

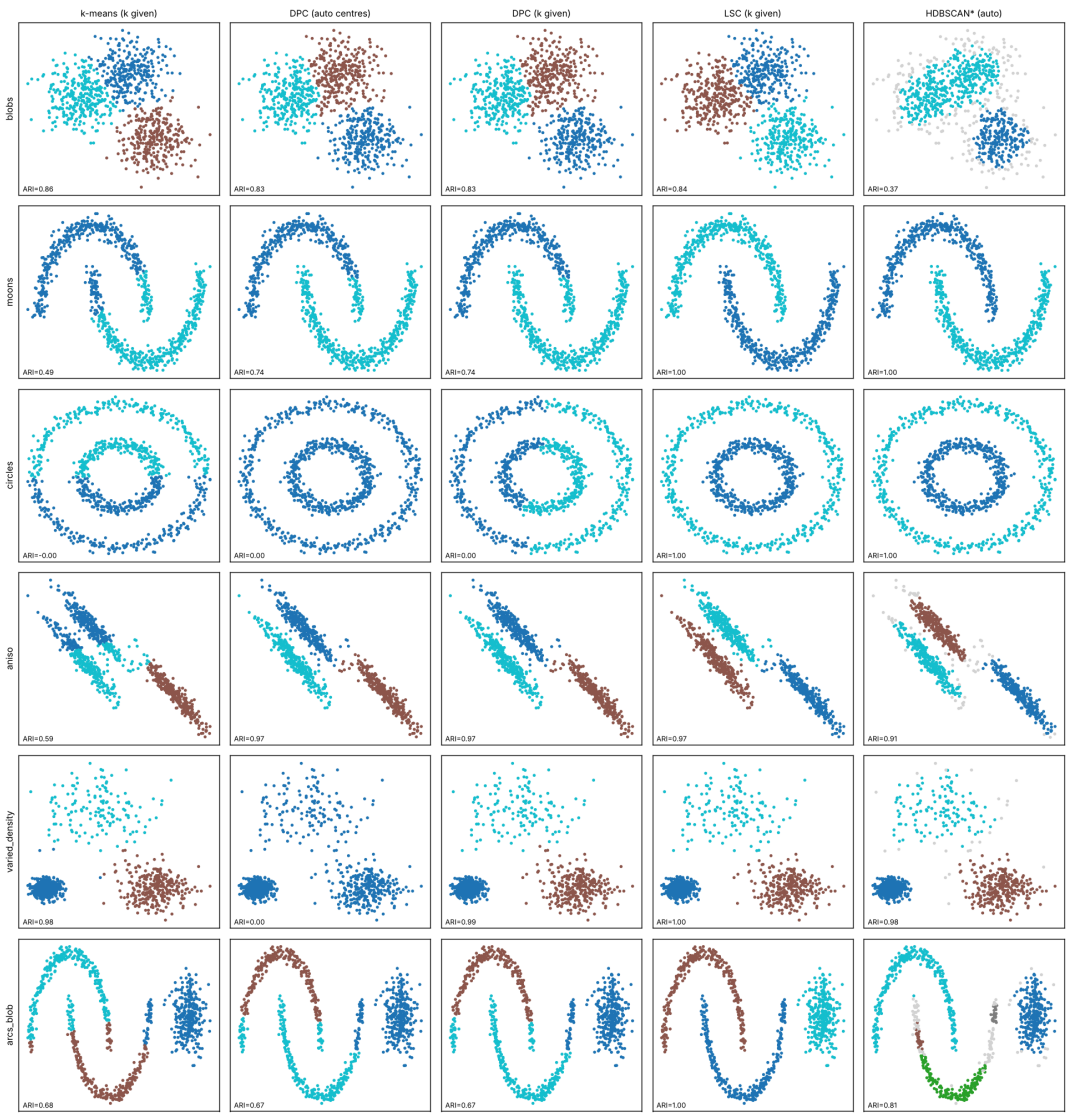


Figure 1. Cluster assignments (grey = HDBSCAN* noise). Rows are data sets, columns are methods.

Observations.

- Landmark spectral clustering and HDBSCAN* recover the non-convex moons and concentric circles, where k-means fails. Spectral clustering needs k; HDBSCAN* does not.
- HDBSCAN* scores only 0.37 on the overlapping blobs: it merges two touching blobs into one and labels many border points as noise, which the ARI penalises. Density-based methods are not a general replacement for k-means on overlapping Gaussian clusters.
- DPC with Gaussian density is competitive on the elongated clusters (0.97) but fails on circles. Its automatic elbow rule picked a single centre on the varied-density set (ARI 0.00) although the same method with k given scored 0.99, so the centre-selection step, not the assignment, was the failure. This agrees with the literature's emphasis on better centre selection and label assignment [5, 6].

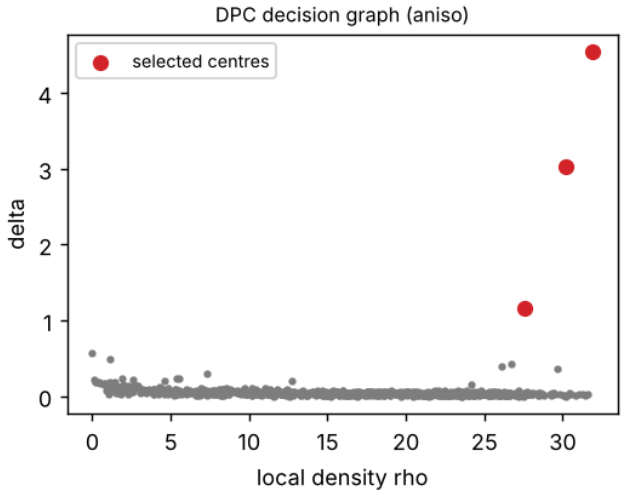


Figure 2. DPC decision graph on the anisotropic set; selected centres in red.

7.2 Deep clustering on digits

Method (5 seeds)	ACC	NMI	ARI
k-means on raw pixels (reference, 1 run)	0.792	0.742	0.666
Autoencoder + centroid DC, no reclustering	0.699 ± 0.047	0.701 ± 0.046	0.584 ± 0.053
+ periodic reclustering	0.699 ± 0.048	0.702 ± 0.046	0.584 ± 0.053
+ soft reset and reclustering (BRB-style)	0.696 ± 0.039	0.710 ± 0.030	0.592 ± 0.046

Table 5. Mean ± standard deviation over five seeds on 1,797 digit images.

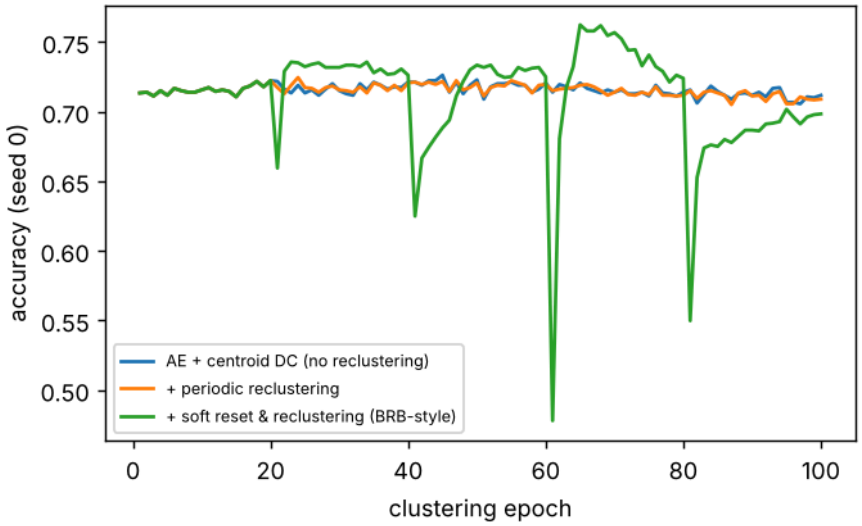


Figure 3. Clustering accuracy per epoch for one seed.

Interpretation. The three deep variants are statistically indistinguishable here (differences are much smaller than the seed-to-seed standard deviation), and all are below plain k-means on raw pixels. That is plausible for a small NumPy autoencoder on a data set where raw pixels already cluster well, and it is not evidence for or against the reclustering barrier reported in [18], whose experiments use much larger pre-trained backbones and image benchmarks. To test BRB properly, use the authors' released implementation.

8. Limitations and recommended next steps

- **Depth of paper review.** Recent papers were reviewed from abstracts and published records, not full texts. Before relying on a claim, read the paper; the foundational algorithms in Section 4 are better established than the recent variants.
- **Coverage.** The search was topical rather than exhaustive. Areas touched only briefly: multi-view and incomplete-view clustering, graph clustering, interpretable clustering, clustering ensembles and streaming clustering.
- **Torque Clustering.** Not reproduced here (Section 4.5). If it matters for your work, obtain the authors' code and check its licence.
- **Experiments.** Small synthetic and toy data; single data-generation seed for 2-D sets; no hyper-parameter tuning beyond defaults shown. The kNN density-peaks variant and the BRB-style model are simplified.
- **Suggested follow-ups.** (a) Run the authors' released code for [3], [18] and [19] on a common set of benchmarks; (b) add DECS or DPAC reference implementations from their repositories; (c) evaluate on larger tabular and image data with repeated seeds.

References

[1]Y. Ren, J. Pu, Z. Yang, J. Xu, G. Li, X. Pu, P. S. Yu, and L. He, “Deep clustering: A comprehensive survey,” IEEE Trans. Neural Netw. Learn. Syst., 2024, doi: 10.1109/TNNLS.2024.3403155.

[2]S. Zhou et al., “A comprehensive survey on deep clustering: Taxonomy, challenges, and future directions,” ACM Comput. Surv., vol. 57, no. 3, art. 69, 2025, doi: 10.1145/3689036.

[3]J. Yang and C.-T. Lin, “Autonomous clustering by fast find of mass and distance peaks,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 47, no. 7, 2025, doi: 10.1109/TPAMI.2025.3535743.

[4]A. Rodriguez and A. Laio, “Clustering by fast search and find of density peaks,” Science, vol. 344, no. 6191, pp. 1492–1496, 2014.

[5]J. Che, W. Zang, J. Xiong, and X. Liu, “Density peaks clustering based on potential model and diffusion strength,” IEEE Access, vol. 10, pp. 128604–128621, 2022, doi: 10.1109/ACCESS.2022.3227936.

[6]S. Zhou, B. Peng, W. Xu, and L. Ren, “SM-DPC: Clustering by fast search and find of density peaks based on SNN with multi-cluster fusion strategy,” IEEE Access, vol. 12, pp. 76413–76431, 2024, doi: 10.1109/ACCESS.2024.3404917.

[7]R. J. G. B. Campello, D. Moulavi, and J. Sander, “Density-based clustering based on hierarchical density estimates,” in Proc. PAKDD, LNCS 7819, 2013, pp. 160–172.

[8]R. J. G. B. Campello, D. Moulavi, A. Zimek, and J. Sander, “Hierarchical density estimates for data clustering, visualization, and outlier detection,” ACM Trans. Knowl. Discov. Data, vol. 10, no. 1, art. 5, 2015.

[9]L. McInnes and J. Healy, “Accelerated hierarchical density based clustering,” in Proc. IEEE Int. Conf. Data Mining Workshops (ICDMW), 2017.

[10]X. Chen and D. Cai, “Large scale spectral clustering with landmark-based representation,” in Proc. AAAI Conf. Artif. Intell., vol. 25, no. 1, 2011, doi: 10.1609/aaai.v25i1.7900.

[11]D. Cai and X. Chen, “Large scale spectral clustering via landmark-based sparse representation,” IEEE Trans. Cybern., 2015, doi: 10.1109/TCYB.2014.2358564.

[12]C.-L. Wang, F. Nie, R. Wang, and X. Li, “Revisiting fast spectral clustering with anchor graph,” in Proc. IEEE ICASSP, 2020.

[13]B. Yang, X. Zhang, J. Wu, F. Nie, F. Wang, and B. Chen, “Scalable min-max multi-view spectral clustering,” IEEE Trans. Knowl. Data Eng., 2025, pp. 2918–2931, doi: 10.1109/TKDE.2025.3543817.

[14]“SC-FAGC: Size constrained fast anchor-based graph clustering,” ICML 2026 (poster; author list not verified), https://icml.cc/virtual/2026/poster/65836.

[15]Z. Cheng, F. Li, J. Wang, and Y. Qian, “Deep embedding clustering driven by sample stability,” in Proc. IJCAI, 2024, pp. 3854–3862.

[16]Y. Yan, N. Lu, and R. Yan, “Deep online probability aggregation clustering,” in Proc. ECCV, 2024, doi: 10.1007/978-3-031-73668-1_3.

[17]A. Kumar, A. Chakrabarty, S. S. Mullick, and S. Das, “Enhancing contrastive clustering with negative pair-guided regularization,” Trans. Mach. Learn. Res., 2024.

[18]L. Miklautz, T. Klein, K. Sidak, C. Leiber, T. Lang, A. Shkabrii, S. Tschiatschek, and C. Plant, “Breaking the reclustering barrier in centroid-based deep clustering,” in Proc. ICLR, 2025.

[19]Z. Li, Y. Jia, H. Liu, and J. Hou, “Learning from sample stability for deep clustering,” in Proc. 42nd ICML, PMLR vol. 267, pp. 34904-34919, 2025.

[20]M. Ronen, S. E. Finder, and O. Freifeld, “DeepDPM: Deep clustering with an unknown number of clusters,” in Proc. IEEE/CVF CVPR, 2022, pp. 9851–9860.

lsc.py

```
"""Landmark-based Spectral Clustering (LSC) for large-scale data.


Chen & Cai, "Large Scale Spectral Clustering with Landmark-Based
Representation", AAAI 2011 (doi:10.1609/aaai.v25i1.7900); extended in
    Cai &
Chen, IEEE Trans. Cybernetics, 2015 (doi:10.1109/TCYB.2014.2358564).


This is the single-view anchor/landmark graph that recent multi-view
    methods
(e.g. SMMSC, IEEE TKDE 2025) build on.


Algorithm (LSC-K variant: k-means landmarks)
1. Pick p << n landmarks U with k-means.
2. For every point x_i keep its r nearest landmarks and set
       z_ij = K(x_i, u_j) / sum_{j' in N_r(i)} K(x_i, u_j'),
   with a Gaussian kernel -> sparse Z (n x p), rows sum to 1.
3. Degree d_i = z_i . (Z^T 1);   Z_hat = D^{-1/2} Z.
4.  The top-k left singular vectors of Z_hat equal the top-k
    eigenvectors of
   the implied n x n normalised affinity Z_hat Z_hat^T (never formed).
5. Row-normalise the embedding and run k-means.
```

```
Cost: O(n p d) for the graph and O(n p k) for the SVD (linear in n).
"""
import numpy as np
from scipy.sparse import csr_matrix
from scipy.sparse.linalg import svds
from sklearn.cluster import KMeans
from sklearn.preprocessing import normalize



class LandmarkSpectral:
            def   __init__(self,   n_clusters,   n_landmarks=200,   r=5,
    random_state=0):
        self.n_clusters = n_clusters
        self.n_landmarks = n_landmarks
        self.r = r
        self.random_state = random_state

    def fit(self, X):
        X = np.asarray(X, dtype=float)
        n = len(X)
        p = min(self.n_landmarks, n)
        km = KMeans(p, n_init=1, random_state=self.random_state).fit(X)
        U = km.cluster_centers_
          # squared distances to landmarks (n x p), keep r nearest per
    point
             d2  =  ((X  **  2).sum(1)[:,  None]  -  2  *  X  @  U.T  +  (U  **
    2).sum(1)[None, :])
        d2 = np.maximum(d2, 0)
        r = min(self.r, p)
        idx = np.argpartition(d2, r - 1, axis=1)[:, :r]
        rows = np.repeat(np.arange(n), r)
        dist2 = d2[np.arange(n)[:, None], idx]
        sigma2 = np.mean(dist2) + 1e-12  # Gaussian bandwidth
        w = np.exp(-dist2 / (2 * sigma2))
        w /= w.sum(axis=1, keepdims=True) + 1e-12
        Z = csr_matrix((w.ravel(), (rows, idx.ravel())), shape=(n, p))
```

```
        deg = Z @ np.asarray(Z.sum(axis=0)).ravel()  # d_i = z_i . (Z^T
    1)
            Zh = csr_matrix(Z.multiply(1.0 / np.sqrt(deg + 1e-12)[:,
    None]))
        k = min(self.n_clusters, p - 1)
        U_svd, s, _ = svds(Zh, k=k, random_state=self.random_state)
        emb = normalize(U_svd[:, np.argsort(-s)])
        self.embedding_ = emb
        self.labels_ = KMeans(self.n_clusters, n_init=10,

    random_state=self.random_state).fit_predict(emb)
        return self

    def fit_predict(self, X):
        return self.fit(X).labels_
```

hdbscan_star.py

```
import numpy as np
from scipy.sparse.csgraph import minimum_spanning_tree
from scipy.spatial.distance import cdist



class HDBSCANStar:
    def __init__(self, min_cluster_size=15, min_samples=None):
        self.min_cluster_size = min_cluster_size
        self.min_samples = min_samples

    def _single_linkage(self, X):
        n = len(X)
        ms = self.min_samples or self.min_cluster_size
        D = cdist(X, X)
        core = np.sort(D, axis=1)[:, min(ms, n) - 1]
        mr = np.maximum(D, np.maximum(core[:, None], core[None, :]))
        np.fill_diagonal(mr, 0)
        mst = minimum_spanning_tree(mr).tocoo()
```

```
        order = np.argsort(mst.data, kind="stable")
        a, b, w = mst.row[order], mst.col[order], mst.data[order]
        parent = np.arange(2 * n - 1)
        size = np.r_[np.ones(n, int), np.zeros(n - 1, int)]
        left = np.full(2 * n - 1, -1)
        right = np.full(2 * n - 1, -1)
        height = np.zeros(2 * n - 1)

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        nxt = n
        for u, v, d in zip(a, b, w):
            ru, rv = find(u), find(v)
            left[nxt], right[nxt], height[nxt] = ru, rv, d
            size[nxt] = size[ru] + size[rv]
            parent[ru] = parent[rv] = nxt
            nxt += 1
        return left, right, height, size, 2 * n - 2

    def fit(self, X):
        X = np.asarray(X, dtype=float)
        n = len(X)
        left, right, height, size, root = self._single_linkage(X)
        mcs = self.min_cluster_size
        lam = lambda d: 1.0 / d if d > 0 else np.inf

        # leaves of every dendrogram node (iterative)
        def leaves(node):
            out, stack = [], [node]
            while stack:
                v = stack.pop()
                if v < n:
```

```
                out.append(v)
            else:
                stack += [left[v], right[v]]
        return out

          clusters = {0: dict(node=root, birth=0.0, fallen=0.0,
end_size=0,
                      end_lam=0.0, kids=[], parent=None)}
    stack = [(root, 0)]
    while stack:
        node, cid = stack.pop()
        C = clusters[cid]
        while True:
            if node < n:
                C["end_size"], C["end_lam"] = 1, C["birth"]
                break
            l, r_ = left[node], right[node]
            lm = lam(height[node])
            sl, sr = size[l], size[r_]
            if sl >= mcs and sr >= mcs:
                C["end_size"], C["end_lam"] = size[node], lm
                for ch in (l, r_):
                    nid = len(clusters)
                            clusters[nid] = dict(node=ch, birth=lm,
fallen=0.0,
                                                 end_size=0, end_lam=0.0,
kids=[],
                                                 parent=cid)
                    C["kids"].append(nid)
                    stack.append((ch, nid))
                break
             if sl < mcs and sr < mcs:  # cluster dies, everything
falls off
                C["fallen"] += size[node] * (lm - C["birth"])
                C["end_size"], C["end_lam"] = 0, lm
                break
```

```
            small, big = (l, r_) if sl < mcs else (r_, l)
            C["fallen"] += size[small] * (lm - C["birth"])
            node = big

    def stability(cid):
        C = clusters[cid]
             end = C["end_size"] * (C["end_lam"] - C["birth"]) if
np.isfinite(C["end_lam"]) else 0.0
        return C["fallen"] + end

    sel_stab, selected = {}, {}
       for cid in sorted(clusters, reverse=True):  # children have
larger ids
        C = clusters[cid]
        s_self = stability(cid)
        if not C["kids"]:
            sel_stab[cid], selected[cid] = s_self, [cid]
            continue
        s_kids = sum(sel_stab[k] for k in C["kids"])
        if cid != 0 and s_self >= s_kids:
            sel_stab[cid], selected[cid] = s_self, [cid]
        else:
            sel_stab[cid] = s_kids
                  selected[cid] = [c for k in C["kids"] for c in
selected[k]]
    chosen = selected[0]
     if chosen == [0]:  # whole data set is one cluster -> all noise
by default
        chosen = []
    labels = np.full(n, -1)
    for lab, cid in enumerate(chosen):
        labels[leaves(clusters[cid]["node"])] = lab
    self.labels_ = labels
    self.n_clusters_ = len(chosen)
    return self
```

```
    def fit_predict(self, X):
        return self.fit(X).labels_
```

metrics.py

```
"""External validity metrics: ARI, NMI and Hungarian-matched clustering
    accuracy."""
import numpy as np
from scipy.optimize import linear_sum_assignment
from sklearn.metrics import adjusted_rand_score,
    normalized_mutual_info_score


def clustering_accuracy(y_true, y_pred):
    """Best one-to-one label matching accuracy (Hungarian
    algorithm)."""
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    classes = np.unique(y_true)
    clusters = np.unique(y_pred)
    w = np.zeros((len(clusters), len(classes)), dtype=np.int64)
    for i, c in enumerate(clusters):
        for j, k in enumerate(classes):
            w[i, j] = np.sum((y_pred == c) & (y_true == k))
    r, c = linear_sum_assignment(-w)
    return w[r, c].sum() / len(y_true)


def evaluate(y_true, y_pred):
    return {
        "ARI": adjusted_rand_score(y_true, y_pred),
        "NMI": normalized_mutual_info_score(y_true, y_pred),
        "ACC": clustering_accuracy(y_true, y_pred),
        "k": int(len(np.unique(y_pred[y_pred >= 0]))),
    }
```

contrastive.py

```
"""Compact Contrastive Clustering (CC) in PyTorch"""

from __future__ import annotations

import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F



def contrastive_loss(a: torch.Tensor, b: torch.Tensor, tau: float) ->
    torch.Tensor:
    m = a.shape[0]
    z = F.normalize(torch.cat([a, b], 0), dim=1)
    sim = z @ z.t() / tau
    sim.fill_diagonal_(float("-inf"))
        target = torch.cat([torch.arange(m, 2 * m), torch.arange(0,
    m)]).to(a.device)
    return F.cross_entropy(sim, target)



class CCNet(nn.Module):
        def __init__(self, in_dim, n_clusters, hidden=256, feat=128,
    proj=64):
        super().__init__()
        self.backbone = nn.Sequential(
            nn.Linear(in_dim, hidden), nn.ReLU(),
            nn.Linear(hidden, feat), nn.ReLU())
             self.instance_head = nn.Sequential(nn.Linear(feat, feat),
    nn.ReLU(), nn.Linear(feat, proj))
              self.cluster_head = nn.Sequential(nn.Linear(feat, feat),
    nn.ReLU(),
                                          nn.Linear(feat, n_clusters))

    def forward(self, x):
```

```
        h = self.backbone(x)
          return self.instance_head(h), F.softmax(self.cluster_head(h),
    dim=1)



class ContrastiveClustering:
        def __init__(self, n_clusters, image_shape=None, epochs=200,
    batch_size=256, lr=3e-3,
                 tau_instance=0.5, tau_cluster=1.0, entropy_weight=1.0,
                                  noise=0.05, mask_prob=0.1, max_shift=1,
    random_state=0, verbose=False):
        self.n_clusters = n_clusters
         self.image_shape = image_shape          # e.g. (8, 8): enables
    random-shift augmentation
        self.epochs, self.batch_size, self.lr = epochs, batch_size, lr
        self.tau_instance, self.tau_cluster = tau_instance, tau_cluster
        self.entropy_weight = entropy_weight
         self.noise, self.mask_prob, self.max_shift = noise, mask_prob,
    max_shift
        self.random_state, self.verbose = random_state, verbose


            #   ------------------------------------------------------------
    augmentation
    def _augment(self, x, gen):
        n = x.shape[0]
        if self.image_shape is not None and self.max_shift > 0:
            h, w = self.image_shape
            s = self.max_shift
            img = F.pad(x.view(n, h, w), (s, s, s, s))
            dy = torch.randint(-s, s + 1, (n,), generator=gen)
            dx = torch.randint(-s, s + 1, (n,), generator=gen)
            ii = torch.arange(h)[None, :] + s + dy[:, None]
            jj = torch.arange(w)[None, :] + s + dx[:, None]
               x = img[torch.arange(n)[:, None, None], ii[:, :, None],
    jj[:, None, :]].reshape(n, -1)
        x = x + self.noise * torch.randn(x.shape, generator=gen)
```

```
                keep = (torch.rand(x.shape, generator=gen) >
self.mask_prob).float()
        return x * keep


    # ---------------------------------------------------------------
fit
    def fit_predict(self, X):
        X = torch.tensor(np.asarray(X, dtype=np.float32))
        torch.manual_seed(self.random_state)
        gen = torch.Generator().manual_seed(self.random_state)
        n, d = X.shape
        K = self.n_clusters
        net = CCNet(d, K)
        opt = torch.optim.Adam(net.parameters(), lr=self.lr)
        self.loss_history_ = []
        for ep in range(self.epochs):
            perm = torch.randperm(n, generator=gen)
            total = 0.0
                    for s in range(0, n - self.batch_size + 1,
self.batch_size):
                xb = X[perm[s:s + self.batch_size]]
                za, ca = net(self._augment(xb, gen))
                zb, cb = net(self._augment(xb, gen))
                l_ins = contrastive_loss(za, zb, self.tau_instance)
                            l_clu = contrastive_loss(ca.t(), cb.t(),
self.tau_cluster)
                pa, pb = ca.mean(0), cb.mean(0)
                   # log K - H(p): zero for uniform cluster usage, > 0
when collapsing
                l_ent = (np.log(K) + (pa * torch.log(pa + 1e-8)).sum()
                                   + np.log(K) + (pb * torch.log(pb +
1e-8)).sum())
                loss = l_ins + l_clu + self.entropy_weight * l_ent
                opt.zero_grad()
                loss.backward()
                opt.step()
                total += loss.item()
```

```
            self.loss_history_.append(total)
            if self.verbose and (ep + 1) % 25 == 0:
                print(f"epoch {ep + 1:4d}  loss {total:.3f}")
        net.eval()
        with torch.no_grad():
            _, c = net(X)
        self.net_ = net
        self.labels_ = c.argmax(1).numpy()
        return self.labels_
```

uspec.py

```
from __future__ import annotations

import numpy as np
import scipy.sparse as sp
from sklearn.cluster import KMeans
from sklearn.metrics import pairwise_distances_argmin_min,
pairwise_distances_argmin
from sklearn.neighbors import NearestNeighbors


def select_representatives(X, p, candidate_factor=10, kmeans_iter=10,
rng=None):
    """Hybrid selection: random candidates -> k-means centres"""
    rng = np.random.default_rng(rng)
    n = len(X)
    p = min(p, n)
    n_cand = min(n, candidate_factor * p)
    cand = X[rng.choice(n, n_cand, replace=False)]
    if n_cand == p:
        return cand.copy()
    km = KMeans(n_clusters=p, n_init=1, max_iter=kmeans_iter,
                random_state=int(rng.integers(1 << 31))).fit(cand)
    return km.cluster_centers_
```

```
def approx_k_nearest_representatives(X, R, K, k_prime_factor=10,
chunk=20000, rng=None):
    """Approximate K nearest representatives for all rows of X"""
    rng = np.random.default_rng(rng)
    p = len(R)
    K = min(K, p)
    n_groups = max(1, int(np.sqrt(p)))
    km = KMeans(n_clusters=n_groups, n_init=1, max_iter=20,
                random_state=int(rng.integers(1 << 31))).fit(R)
    rep_group = km.labels_
    k_prime = min(p, max(K, k_prime_factor * K))
                                                rep_nbrs =
NearestNeighbors(n_neighbors=k_prime).fit(R).kneighbors(R,
return_distance=False)


    n = len(X)
    idx_out = np.empty((n, K), dtype=np.int64)
    dist_out = np.empty((n, K))
    for s in range(0, n, chunk):
        Xc = X[s:s + chunk]
         g = pairwise_distances_argmin(Xc, km.cluster_centers_)        #
nearest rep-group
        nearest_rep = np.empty(len(Xc), dtype=np.int64)
                                    for gid in np.unique(g):
# nearest rep in group
            rows = np.where(g == gid)[0]
            members = np.where(rep_group == gid)[0]
                                           if len(members) == 0:
# empty group guard
                members = np.arange(p)
            j, _ = pairwise_distances_argmin_min(Xc[rows], R[members])
            nearest_rep[rows] = members[j]
                                    cand = rep_nbrs[nearest_rep]
# (m, K')
                            diff = Xc[:, None, :] - R[cand]
# (m, K', d)
        d = np.sqrt((diff ** 2).sum(-1))
```

```
        order = np.argsort(d, axis=1)[:, :K]
        idx_out[s:s + chunk] = np.take_along_axis(cand, order, axis=1)
        dist_out[s:s + chunk] = np.take_along_axis(d, order, axis=1)
    return idx_out, dist_out


def gaussian_bipartite_graph(idx, dist, p):
    """Sparse n x p affinity matrix with Gaussian weights."""
    n, K = idx.shape
    sigma = max(float(dist.mean()), 1e-12)
    w = np.exp(-(dist ** 2) / (2 * sigma ** 2))
    rows = np.repeat(np.arange(n), K)
      return sp.csr_matrix((w.ravel(), (rows, idx.ravel())), shape=(n,
p))


def transfer_cut(B, k, random_state=0, n_init=10):
     """Partition the bipartite graph given by sparse B (n x p) into k
groups.

    Solves L_Y u = lambda D_Y u on the p-node graph E_Y = B^T D_X^-1 B,
then
    transfers u to objects: v = D_X^-1 B u / (1 - lambda).
    """
    dx = np.asarray(B.sum(axis=1)).ravel()
    dy = np.asarray(B.sum(axis=0)).ravel()
    dx = np.maximum(dx, 1e-12)
    dy = np.maximum(dy, 1e-12)
    Dx_inv_B = sp.diags(1.0 / dx) @ B
    Ey = (B.T @ Dx_inv_B).toarray()
    dy_isqrt = 1.0 / np.sqrt(dy)
    M = (dy_isqrt[:, None] * Ey) * dy_isqrt[None, :]
    M = (M + M.T) / 2
    k = min(k, M.shape[0])
    evals, evecs = np.linalg.eigh(M)
    top = np.argsort(evals)[::-1][:k]
```

```
    gamma = np.maximum(evals[top], 1e-8)                    # gamma = 1 -
lambda
    U = dy_isqrt[:, None] * evecs[:, top]
    V = (Dx_inv_B @ U) / gamma[None, :]
    V = np.asarray(V)
                    km     =     KMeans(n_clusters=k,     n_init=n_init,
random_state=random_state).fit(V)
    return km.labels_



class USPEC:
    def __init__(self, n_clusters, n_representatives=1000, knn=5,
                 candidate_factor=10, random_state=0):
        self.n_clusters = n_clusters
        self.n_representatives = n_representatives
        self.knn = knn
        self.candidate_factor = candidate_factor
        self.random_state = random_state

    def fit_predict(self, X):
        X = np.asarray(X, dtype=float)
        rng = np.random.default_rng(self.random_state)
                R  =  select_representatives(X,  self.n_representatives,
self.candidate_factor, rng=rng)
          idx, dist = approx_k_nearest_representatives(X, R, self.knn,
rng=rng)
        B = gaussian_bipartite_graph(idx, dist, len(R))
                    self.labels_   =   transfer_cut(B,   self.n_clusters,
random_state=self.random_state)
        return self.labels_



class USENC:
       def  __init__(self,  n_clusters,  n_base=10,  k_range=(20,  60),
n_representatives=1000,
                 knn=5, random_state=0):
        self.n_clusters = n_clusters
```

```
        self.n_base = n_base
        self.k_range = k_range
        self.n_representatives = n_representatives
        self.knn = knn
        self.random_state = random_state

    def fit_predict(self, X):
        X = np.asarray(X, dtype=float)
        rng = np.random.default_rng(self.random_state)
        n = len(X)
        lo, hi = self.k_range
        lo = max(lo, self.n_clusters)
        hi = max(hi, lo)
        cols, offset = [], 0
        rows = []
        for m in range(self.n_base):
            k_i = int(rng.integers(lo, hi + 1))
            base = USPEC(k_i, self.n_representatives, self.knn,
                                    random_state=int(rng.integers(1 << 31))).fit_predict(X)
                _, base = np.unique(base, return_inverse=True)          # remove empty ids
            rows.append(np.arange(n))
            cols.append(base + offset)
            offset += base.max() + 1
        rows, cols = np.concatenate(rows), np.concatenate(cols)
         B = sp.csr_matrix((np.ones(len(rows)), (rows, cols)), shape=(n, offset))
        self.base_cluster_count_ = offset
                    self.labels_ = transfer_cut(B, self.n_clusters, random_state=self.random_state)
        return self.labels_
```

__init__.py

```
from .deep_brb import DeepCentroidClustering
```

```
from .dpc import DensityPeaks
from .hdbscan_star import HDBSCANStar
from .lsc import LandmarkSpectral

__all__ = ["DensityPeaks", "LandmarkSpectral", "HDBSCANStar",
"DeepCentroidClustering"]
```

run_demo.py

```
import argparse
import os
import warnings

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans

from clustering.datasets import digits_data, two_dim_suite
from clustering.deep_brb import DeepCentroidClustering
from clustering.dpc import DensityPeaks
from clustering.hdbscan_star import HDBSCANStar
from clustering.lsc import LandmarkSpectral
from clustering.metrics import evaluate

warnings.filterwarnings("ignore")
OUT = os.path.join(os.path.dirname(os.path.abspath(__file__)),
    "results")
os.makedirs(OUT, exist_ok=True)



def run_2d():
    suite = two_dim_suite()
    methods = {
```

```
          "k-means (k given)": lambda X, k: KMeans(k, n_init=10,
random_state=0).fit_predict(X),
                      "DPC (auto centres)": lambda X, k:
DensityPeaks(density="gaussian").fit_predict(X),
          "DPC (k given)": lambda X, k: DensityPeaks(n_clusters=k,
density="gaussian").fit_predict(X),
              "LSC (k given)": lambda X, k: LandmarkSpectral(k,
n_landmarks=100, r=5).fit_predict(X),
    "HDBSCAN* (auto)": lambda X, k: HDBSCANStar(15).fit_predict(X),
}
rows, preds = [], {}
for dname, (X, y) in suite.items():
    k = len(np.unique(y))
    for mname, f in methods.items():
        lab = f(X, k)
        preds[(dname, mname)] = lab
         rows.append(dict(dataset=dname, method=mname, **evaluate(y,
lab)))
df = pd.DataFrame(rows)
df.to_csv(os.path.join(OUT, "results_2d.csv"), index=False)
           piv = df.pivot(index="dataset", columns="method",
values="ARI")[list(methods)]
        print("\nAdjusted Rand Index (higher is better)\n",
piv.round(2).to_string())
piv.round(3).to_csv(os.path.join(OUT, "results_2d_ari_table.csv"))

# figure: datasets x methods
  fig, axes = plt.subplots(len(suite), len(methods), figsize=(3 *
len(methods), 2.6 * len(suite)))
for i, (dname, (X, y)) in enumerate(suite.items()):
    for j, mname in enumerate(methods):
        ax = axes[i, j]
        lab = preds[(dname, mname)]
        noise = lab < 0
          ax.scatter(X[~noise, 0], X[~noise, 1], c=lab[~noise], s=4,
cmap="tab10")
        ax.scatter(X[noise, 0], X[noise, 1], c="lightgrey", s=4)
```

```
            ax.set_xticks([]); ax.set_yticks([])
            if i == 0:
                ax.set_title(mname, fontsize=9)
            if j == 0:
                ax.set_ylabel(dname, fontsize=9)
                ax.text(0.02, 0.02, f"ARI={df[(df.dataset == dname) &
    (df.method == mname)].ARI.iloc[0]:.2f}",
                    transform=ax.transAxes, fontsize=7)
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, "fig_2d.png"), dpi=130)
    plt.close(fig)

    # DPC decision graph example
    X, y = suite["aniso"]
    d = DensityPeaks(density="gaussian").fit(X)
    fig, ax = plt.subplots(figsize=(4.2, 3.4))
    ax.scatter(d.rho_, d.delta_, s=6, c="grey")
          ax.scatter(d.rho_[d.centers_], d.delta_[d.centers_], s=40,
    c="tab:red", label="selected centres")
        ax.set_xlabel("local density rho"); ax.set_ylabel("delta");
    ax.legend(fontsize=8)
    ax.set_title("DPC decision graph (aniso)", fontsize=9)
                fig.tight_layout();    fig.savefig(os.path.join(OUT,
    "fig_dpc_decision_graph.png"), dpi=130)
    plt.close(fig)



def run_deep(seeds=5):
    X, y = digits_data()
    rows, curves = [], {}
                km    =    evaluate(y,    KMeans(10,    n_init=10,
    random_state=0).fit_predict(X))
    rows.append(dict(method="k-means on raw pixels", seed="-", **km))
    for mode, name in [("none", "AE + centroid DC (no reclustering)"),
                       ("recluster", "+ periodic reclustering"),
                                ("brb", "+ soft reset & reclustering
    (BRB-style)")]:
```

```
        for s in range(seeds):
            m = DeepCentroidClustering(10, mode=mode, random_state=s)
            m.fit(X, y_monitor=y)
                    rows.append(dict(method=name, seed=s, **evaluate(y,
    m.labels_)))
            if s == 0:
                curves[name] = m.history_
    df = pd.DataFrame(rows)
    df.to_csv(os.path.join(OUT, "results_deep_runs.csv"), index=False)
          g = df[df.seed != "-"].groupby("method")[["ACC", "NMI",
    "ARI"]].agg(["mean", "std"])
    g.to_csv(os.path.join(OUT, "results_deep_summary.csv"))
     print(f"\nDigits (1797 x 64), mean +- std over {seeds} seeds\n",
    g.round(3).to_string())
     print("\nk-means on raw pixels:", {k: round(float(v), 3) for k, v
    in km.items()})
    fig, ax = plt.subplots(figsize=(5.4, 3.4))
    for name, h in curves.items():
        ax.plot(range(1, len(h) + 1), h, label=name)
      ax.set_xlabel("clustering epoch"); ax.set_ylabel("accuracy (seed
    0)")
    ax.legend(fontsize=7); fig.tight_layout()
    fig.savefig(os.path.join(OUT, "fig_deep_curve.png"), dpi=130)
    plt.close(fig)


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
           ap.add_argument("--only", choices=["2d", "deep", "all"],
    default="all")
    ap.add_argument("--seeds", type=int, default=5)
    a = ap.parse_args()
    if a.only in ("2d", "all"):
        run_2d()
    if a.only in ("deep", "all"):
        run_deep(a.seeds)
    print("\nSaved to", OUT)
```

datasets.py

```
"""Benchmark data sets used by the demos (all generated locally)."""
import numpy as np
from sklearn import datasets
from sklearn.preprocessing import StandardScaler



def two_dim_suite(n=900, seed=0):
    """Classic 2-D shape benchmarks. Returns {name: (X, y)}."""
    rs = np.random.RandomState(seed)
    out = {}

    X, y = datasets.make_blobs(n_samples=n, centers=3, cluster_std=0.9,
                               random_state=seed)
    out["blobs"] = (X, y)

            X,   y   =   datasets.make_moons(n_samples=n,   noise=0.06,
random_state=seed)
    out["moons"] = (X, y)

    X, y = datasets.make_circles(n_samples=n, factor=0.45, noise=0.05,
                                 random_state=seed)
    out["circles"] = (X, y)

            X,   y   =   datasets.make_blobs(n_samples=n,   centers=3,
random_state=170)
    X = X @ np.array([[0.6, -0.64], [-0.4, 0.85]])
    out["aniso"] = (X, y)

    X, y = datasets.make_blobs(n_samples=[n // 2, n // 3, n - n // 2 -
n // 3],
                               centers=[[0, 0], [6, 0], [3, 6]],
                                        cluster_std=[0.4, 1.0, 1.6],
random_state=seed)
    out["varied_density"] = (X, y)
```

```
    # two dense spirals-like arcs + a blob (non-convex mixture)
    t = rs.uniform(0, np.pi, n // 3)
     a = np.c_[np.cos(t), np.sin(t)] * 3 + rs.normal(0, 0.12, (n // 3,
2))
     b = np.c_[1 - np.cos(t), 0.5 - np.sin(t)] * 3 + rs.normal(0, 0.12,
(n // 3, 2))
    c = rs.normal([9, 1], 0.6, (n - 2 * (n // 3), 2))
    X = np.vstack([a, b, c])
     y = np.r_[np.zeros(n // 3), np.ones(n // 3), 2 * np.ones(n - 2 * (n
// 3))].astype(int)
    out["arcs_blob"] = (X, y)

     return {k: (StandardScaler().fit_transform(v[0]), v[1]) for k, v in
out.items()}

def digits_data():
    """sklearn 8x8 digits (1797 x 64), scaled to [0, 1]."""
    X, y = datasets.load_digits(return_X_y=True)
    return X / 16.0, y
```

deepbrb.py

```
import numpy as np
from sklearn.cluster import KMeans

from .metrics import clustering_accuracy



class _MLP:
    def __init__(self, sizes, rng):
        self.sizes = sizes
        self.W, self.b = [], []
        for i, o in zip(sizes[:-1], sizes[1:]):
            self.W.append(rng.normal(0, np.sqrt(2.0 / i), (i, o)))
            self.b.append(np.zeros(o))
        self.mW = [np.zeros_like(w) for w in self.W]
```

```
        self.vW = [np.zeros_like(w) for w in self.W]
        self.mb = [np.zeros_like(b) for b in self.b]
        self.vb = [np.zeros_like(b) for b in self.b]
        self.t = 0

    def forward(self, x):
        hs, pre = [x], []
        h = x
        for l, (W, b) in enumerate(zip(self.W, self.b)):
            a = h @ W + b
            pre.append(a)
            h = a if l == len(self.W) - 1 else np.maximum(a, 0)
            hs.append(h)
        return h, (hs, pre)

    def backward(self, g, cache):
        hs, pre = cache
        gW, gb = [None] * len(self.W), [None] * len(self.W)
        for l in reversed(range(len(self.W))):
            if l != len(self.W) - 1:
                g = g * (pre[l] > 0)
            gW[l] = hs[l].T @ g
            gb[l] = g.sum(0)
            g = g @ self.W[l].T
        return gW, gb, g

    def adam(self, gW, gb, lr, b1=0.9, b2=0.999, eps=1e-8):
        self.t += 1
        for l in range(len(self.W)):
            for p, g, m, v in ((self.W, gW, self.mW, self.vW),
                               (self.b, gb, self.mb, self.vb)):
                m[l] = b1 * m[l] + (1 - b1) * g[l]
                v[l] = b2 * v[l] + (1 - b2) * g[l] ** 2
                mh = m[l] / (1 - b1 ** self.t)
                vh = v[l] / (1 - b2 ** self.t)
                p[l] -= lr * mh / (np.sqrt(vh) + eps)
```

```
    def soft_reset(self, lam, rng):
                    """Shrink-and-perturb:  theta  <-  lam*theta  +
(1-lam)*fresh_init."""
                for  l,  (i,  o)  in  enumerate(zip(self.sizes[:-1],
self.sizes[1:])):
            fresh = rng.normal(0, np.sqrt(2.0 / i), (i, o))
            self.W[l] = lam * self.W[l] + (1 - lam) * fresh
            self.b[l] = lam * self.b[l]
        # Adam statistics refer to the old weights -> restart them
        for l in range(len(self.W)):
            self.mW[l][:] = 0; self.vW[l][:] = 0
            self.mb[l][:] = 0; self.vb[l][:] = 0
        self.t = 0


class DeepCentroidClustering:
        def  __init__(self,  n_clusters,  latent=10,  hidden=(128,  64),
mode="brb",
                pretrain_epochs=60, epochs=100, recluster_every=20,
                            beta=1.0,  lam=0.8,  lr=1e-3,  batch=128,
random_state=0):
        assert mode in ("none", "recluster", "brb")
        self.n_clusters = n_clusters
        self.latent = latent
        self.hidden = tuple(hidden)
        self.mode = mode
        self.pretrain_epochs = pretrain_epochs
        self.epochs = epochs
        self.recluster_every = recluster_every
        self.beta = beta
        self.lam = lam
        self.lr = lr
        self.batch = batch
        self.random_state = random_state
```

```
    def _embed(self, X):
        return self.enc.forward(X)[0]

    def _kmeans(self, Z):
                          km   =   KMeans(self.n_clusters,   n_init=10,
random_state=self.random_state).fit(Z)
        return km.cluster_centers_

    def _step(self, xb, C, w_cluster):
        z, ce = self.enc.forward(xb)
        rec, cd = self.dec.forward(z)
        B, d = xb.shape
        g_rec = 2.0 * (rec - xb) / (B * d)
        gWd, gbd, gz = self.dec.backward(g_rec, cd)
        if w_cluster > 0:
              a = np.argmin(((z[:, None, :] - C[None]) ** 2).sum(-1),
axis=1)
            gz = gz + w_cluster * 2.0 * (z - C[a]) / (B * z.shape[1])
        gWe, gbe, _ = self.enc.backward(gz, ce)
        self.dec.adam(gWd, gbd, self.lr)
        self.enc.adam(gWe, gbe, self.lr)

    def fit(self, X, y_monitor=None):
        X = np.asarray(X, dtype=float)
        rng = np.random.RandomState(self.random_state)
        d = X.shape[1]
        self.enc = _MLP([d, *self.hidden, self.latent], rng)
        self.dec = _MLP([self.latent, *self.hidden[::-1], d], rng)
        n = len(X)
        self.history_ = []

        def batches():
            perm = rng.permutation(n)
            for s in range(0, n, self.batch):
                yield X[perm[s:s + self.batch]]
```

```
            for _ in range(self.pretrain_epochs):   # 1) autoencoder
pre-training
            for xb in batches():
                self._step(xb, None, 0.0)

        C = self._kmeans(self._embed(X))  # 2) initial clustering
        for ep in range(1, self.epochs + 1):
            if (self.mode != "none" and ep > 1
                    and (ep - 1) % self.recluster_every == 0):
                if self.mode == "brb":  # soft reset, then recluster
                    self.enc.soft_reset(self.lam, rng)
                C = self._kmeans(self._embed(X))
            for xb in batches():
                self._step(xb, C, self.beta)
              Z = self._embed(X)   # centroid update (closed form, per
epoch)
              a = np.argmin(((Z[:, None, :] - C[None]) ** 2).sum(-1),
axis=1)
            for j in range(self.n_clusters):
                if np.any(a == j):
                    C[j] = Z[a == j].mean(0)
            if y_monitor is not None:
                self.history_.append(clustering_accuracy(y_monitor, a))
        self.centers_ = C
        Z = self._embed(X)
        self.embedding_ = Z
             self.labels_ = np.argmin(((Z[:, None, :] - C[None]) **
2).sum(-1), axis=1)
        return self

    def fit_predict(self, X, y_monitor=None):
        return self.fit(X, y_monitor).labels_
```

dpc.py

```
import numpy as np
```

```
from scipy.spatial.distance import cdist


class DensityPeaks:
    """Parameters
    n_clusters : int or None
        None -> centres chosen automatically with an elbow rule on
                gamma = rho*delta (largest ratio gap among the top
``max_clusters+1``
        points). An int picks the top-k gamma points.
    density : {"cutoff", "gaussian", "knn"}"""


            def __init__(self, n_clusters=None, density="gaussian",
dc_percent=2.0,
                 k=10, max_clusters=15):
        self.n_clusters = n_clusters
        self.density = density
        self.dc_percent = dc_percent
        self.k = k
        self.max_clusters = max_clusters


    def _density(self, D):
        n = D.shape[0]
        iu = np.triu_indices(n, 1)
        if self.density == "knn":
            kk = min(self.k, n - 1)
            knn = np.sort(D, axis=1)[:, 1:kk + 1]
            return np.exp(-np.mean(knn ** 2, axis=1))
        dc = np.percentile(D[iu], self.dc_percent)
        self.dc_ = dc
        if self.density == "cutoff":
            return (D < dc).sum(axis=1) - 1.0
        return np.exp(-(D / dc) ** 2).sum(axis=1) - 1.0


    def fit(self, X):
        X = np.asarray(X, dtype=float)
        n = len(X)
```

```
        D = cdist(X, X)
        rho = self._density(D)
        order = np.argsort(-rho, kind="stable")
        delta = np.zeros(n)
        nn_higher = np.full(n, -1)
        delta[order[0]] = D[order[0]].max()
        for r in range(1, n):
            i = order[r]
            higher = order[:r]
            j = higher[np.argmin(D[i, higher])]
            delta[i] = D[i, j]
            nn_higher[i] = j
        # decision-graph score
        rn = (rho - rho.min()) / (np.ptp(rho) + 1e-12)
        dn = (delta - delta.min()) / (np.ptp(delta) + 1e-12)
        gamma = rn * dn
        if self.n_clusters is None:
               # elbow rule: the largest ratio gap among the top-gamma
points
            top = np.argsort(-gamma)[:min(self.max_clusters + 1, n)]
            g = gamma[top]
            if len(g) < 2:
                kk = 1
            else:
                ratio = g[:-1] / (g[1:] + 1e-12)
                kk = int(np.argmax(ratio)) + 1
            centres = top[:kk]
        else:
            centres = np.argsort(-gamma)[:self.n_clusters]
        labels = np.full(n, -1)
        labels[centres] = np.arange(len(centres))
        for i in order:
            if labels[i] == -1:
                labels[i] = labels[nn_higher[i]]
        self.labels_ = labels
        self.centers_ = centres
```

```
        self.rho_, self.delta_, self.gamma_ = rho, delta, gamma
        self.n_clusters_ = len(centres)
        return self

    def fit_predict(self, X):
        return self.fit(X).labels_
```

Results

https://drive.google.com/drive/folders/1HGmYG0zf-HJHLlPznhiJ_Tjh2FnNvLWb?usp=sharing

Anomaly Detection: A Literature Survey with Implementations

1. Introduction

Anomaly detection finds observations that do not conform to expected behaviour; Chandola, Banerjee and Kumar's 2009 survey [1] remains the standard framing, and it traces the idea back to Edgeworth (1887). This document surveys techniques that approach the problem through **clustering / density** (describe where normal data lives, flag what is far from it) and through **classification / model-based learning** (learn a boundary, a score or a reconstruction that separates normal from abnormal). Each method is explained, supported by papers, and implemented in runnable Python (Section 6).

Types of anomaly:

- **Point anomalies**: a single instance is abnormal relative to all data (a fraudulent transaction amount).
- **Contextual anomalies**: abnormal only in context, such as 30 °C in January; common in time series.
- **Collective anomalies**: a group of instances is abnormal together even if each member looks normal, for example a burst of identical network packets. The clustered "local" anomalies in our synthetic data (Section 6) are a small-scale version of this and are what makes several detectors fail.

Learning settings. The setting decides which family of methods applies:

Setting	Training labels	Typical methods
Unsupervised	none; data assumed mostly normal	k-means distance, GMM, DBSCAN, LOF, Isolation Forest, ECOD
Semi-supervised / one-class	normal only (or a few labelled anomalies)	One-Class SVM, SVDD, autoencoders, Deep SVDD, Deep SAD
Supervised	normal and anomalous	Random Forest, gradient boosting, neural classifiers on imbalanced data

Clustering vs classification framing. A clustering view asks *how well does this point fit a group of similar points?*, so the anomaly score is a distance, a density ratio or a membership probability. A classification view asks *on which side of a learned boundary does this point fall?*, so the score is a decision value, a path length or a reconstruction error. In practice the two blur: DBSCAN is a clustering algorithm whose noise label is a detector, and One-Class SVM is a "classifier" that never sees an anomaly.

A warning from the largest benchmark to date: ADBench ran 30 algorithms on 57 datasets (98,436 experiments) and found that none of the unsupervised methods was statistically better than the others, so algorithm choice depends on the data [2]. The same study found that with only 1% labelled anomalies, most semi-supervised methods beat the best unsupervised one [2]. Our own results in Section 6 are consistent with both points.

2. Taxonomy of techniques

Chandola et al. organise the field by the assumption each technique makes about normal data [1]; later deep-learning surveys keep that logic and add reconstruction-, prediction- and hybrid categories [3][4][5]. The table maps the methods implemented here onto that structure.

Family	Core assumption	Methods covered	Score	Main weakness
Distance / clustering	Normal points lie near a cluster centre or in a dense cluster	k-means distance, GMM, DBSCAN, kNN distance	Distance, neg. log-likelihood, noise flag	Needs cluster count or radius; fails on clusters of unequal density
Density (local)	Normal points have density similar to their neighbours	LOF	Ratio of neighbour density to own density	Sensitive to k; quadratic cost without indexes
Statistical / marginal	Anomalies sit in distribution tails	ECOD	Summed tail probabilities	Blind to anomalies inside the marginal range
One-class classification	Normal data fills a compact region in feature space	One-Class SVM, SVDD, Deep SVDD	Distance to boundary or centre	Kernel and nu tuning; poor scaling
Isolation	Anomalies are few and different, so random cuts isolate them quickly	Isolation Forest, Extended IF	Short average path length	Axis-aligned bias; weak on clustered anomalies
Reconstruction	Normal data compresses well; anomalies do not	Autoencoders, Anomaly Transformer (time series)	Reconstruction error	Over-generalises and reconstructs anomalies too
Supervised classification	Labelled anomalies are representative	Random Forest, boosted trees, Deep SAD (partly)	Class probability	Only finds anomaly types seen in training

Two cross-cutting distinctions matter when reading the literature. First, **global** versus **local** anomalies: a point may be unremarkable globally yet sparse relative to its own neighbourhood, which is the case LOF was built for [6]. Second, **shallow** versus **deep** methods: deep approaches learn features for high-dimensional or structured data (images, sequences) but add training cost and tuning; ADBench benchmarks both shallow and deep detectors and reports no universally best method [2].

3. Clustering and density-based methods

These methods model where normal data concentrates and score each point by how poorly it fits. Chandola et al. file them under the assumption that normal instances belong to a cluster or dense region while anomalies do not, or lie far from the nearest centroid [1].

3.1 k-means distance to centroid

Principle Fit k-means on the training data, then score a point by the distance to its nearest centroid. Points far from every centroid are anomalous.

$$s(x) = \min_{j \in \{1..k\}} \lVert x - \mu_j \rVert_2$$

Strengths and limits It is fast and easy to explain. It assumes spherical clusters of similar spread, so a point in the sparse tail of a wide cluster scores higher than a true anomaly near a tight cluster. In our synthetic data it reaches a mean ROC-AUC of 0.905 against 0.965 for GMM.

3.2 Gaussian mixture model likelihood

Principle. Fit a mixture of k Gaussians by expectation-maximisation and score a point by its negative log-likelihood. Unlike k-means, each component has its own covariance, so a wide cluster tolerates larger distances than a tight one.

$$s(x) = -\log \sum_{j=1}^{k} \pi_j \, \mathcal{N}(x \mid \mu_j, \Sigma_j)$$

Strengths and limits. It handles clusters of unequal density and gives a probabilistic score, but it needs k and a roughly Gaussian shape. It was the best unsupervised method on the synthetic data, which is built from two Gaussians, so that result flatters it. Code: `GMMScore`.

3.3 DBSCAN

Literature. Ester, Kriegel, Sander and Xu introduced DBSCAN at KDD 1996 as a density-based clustering algorithm for clusters of arbitrary shape that needs few parameters [7]. It won the 2014 KDD Test of Time Award.

Principle. A point is a *core point* if at least `MinPts` points lie within radius `eps`. Clusters are maximal sets of density-connected core points plus their border points. Points reachable from no core point are labelled **noise**, and the noise set is the anomaly set.

Our adaptation. DBSCAN returns a binary label, which cannot be ranked or scored by ROC-AUC. We use the distance from a query point to its nearest core point as a continuous score (zero for points inside dense regions).

Strengths and limits. It finds arbitrary-shaped clusters and needs no cluster count. A single global `eps` cannot serve clusters of very different density, which is why it scored only 0.767 ROC-AUC on the synthetic data but 0.943 on the 30-dimensional breast-cancer data where one scale suits the standardised features.

3.4 kNN distance

Principle. Score a point by its distance to its k-th nearest neighbour. It is the simplest distance-based detector and a useful baseline; it ignores local density differences. Code: `KNNDist`.

3.5 Local Outlier Factor (LOF)

Literature. Breunig, Kriegel, Ng and Sander (SIGMOD 2000) assign each object a degree of outlierness that depends on how isolated it is relative to its surrounding neighbourhood, and show it finds outliers that global distance-based methods cannot [6].

Principle. Let $N_k(p)$ be the k nearest neighbours of p. The reachability distance smooths small distances, the local reachability density (lrd) is the inverse of the mean reachability distance, and LOF is the average ratio of the neighbours' density to the point's own density.

$$\text{reach-dist}_k(p,o)=\max\{\,k\text{-dist}(o),\ d(p,o)\,\}$$

$$\qquad$$

$$\text{lrd}_k(p)=\frac{|N_k(p)|}{\sum_{o\in N_k(p)}\text{reach-dist}_k(p,o)}$$

$$\qquad$$

$$\text{LOF}_k(p)=\frac{1}{|N_k(p)|}\sum_{o\in N_k(p)}\frac{\text{lrd}_k(o)}{\text{lrd}_k(p)}$$

A value near 1 means the point is as dense as its neighbours; values well above 1 mean it is locally sparse. scikit-learn's documentation describes LOF the same way [24].

Strengths and limits. LOF handles clusters of different density and had the highest synthetic-data score among distance and density methods (0.957 mean ROC-AUC). It depends on k and costs $O(n^2)$ without a spatial index. It can also be masked by anomalies that form their own small group: against our clustered anomalies its ROC-AUC is 0.457, about chance (Section 6.4). Code: `LOFScratch` (about 25 lines of NumPy) and `LOFSk`. The two give identical rankings in our runs (Spearman 1.000).

4. Classification-based and model-based methods

Here the detector learns a decision function, a tree ensemble or a reconstruction model instead of an explicit cluster structure.

4.1 One-Class SVM

Literature. Schölkopf, Platt, Shawe-Taylor, Smola and Williamson (Neural Computation, 2001) estimate a "simple" region S of input space so that the probability of a test point falling outside S is bounded by a preset value; the method is a natural extension of the support vector algorithm to unlabelled data [8].

Principle. Map data into a kernel feature space and separate it from the origin with maximum margin. The parameter `nu` in (0, 1] is the intended upper bound on the fraction of training points treated as outliers.

Strengths and limits. It needs no anomaly labels and handles non-linear boundaries through the RBF kernel. Kernel methods scale at least quadratically in sample count and degrade on high-dimensional data, which motivated Deep SVDD [13]. In our runs it was sensitive to the data draw (synthetic ROC-AUC 0.861 ± 0.093 over 10 seeds, the largest spread of any method) and weaker than tree or density methods on the 30-dimensional data (0.910).

4.2 Isolation Forest and Extended Isolation Forest

Literature. Liu, Ting and Zhou proposed Isolation Forest at ICDM 2008 as a model-based method that explicitly isolates anomalies instead of profiling normal points, using no distance or density measure [9]. The 2012 TKDD journal version shows linear time complexity with low memory, robustness to swamping and masking, and better AUC than ORCA, One-Class SVM, LOF and Random Forests in their experiments [10]. Hariri, Carrasco Kind and Brunner (Extended Isolation Forest) replace axis-parallel

cuts with random-slope hyperplane cuts to remove the artefacts that axis-aligned splits leave in the score map [11].

Principle. Anomalies are few and different, so random partitioning separates them in fewer splits. Build t trees on sub-samples of size psi (default 256), choosing at each node a random feature and a random split value between its minimum and maximum. The score uses the average path length E[h(x)], normalised by the expected path length of an unsuccessful binary-search-tree lookup c(psi).

Here H(i) is the harmonic number, approximately ln(i) + 0.5772. Scores near 1 indicate anomalies; scores well below 0.5 indicate normal points.

Strengths and limits. It is fast, has few parameters and was the best unsupervised method on the 30-dimensional breast-cancer data (ROC-AUC 0.963 ± 0.018). The score map in Section 6 shows its weakness: axis-parallel cuts create the horizontal and vertical bands visible around the clusters, which is the artefact Extended Isolation Forest was designed to remove [11]. Our scratch version tracks the scikit-learn ranking closely (Spearman 0.993 on synthetic data, 0.935 on breast cancer) but not exactly, because tree randomness and scikit-learn's implementation, which builds on extra-trees regressors [25], differ.

4.3 ECOD (empirical cumulative distribution outlier detection)

Literature. Li, Zhao, Hu, Botta, Ionescu and Chen (IEEE TKDE, 2023) estimate each dimension's empirical CDF, convert it to tail probabilities and aggregate them into one score; the method is parameter-free, interpretable and outperformed 11 baselines on 30 benchmark datasets in the authors' experiments [12].

Principle. For each dimension j compute left and right tail probabilities from the empirical CDF, take the negative log, and sum across dimensions. A skewness-corrected term chooses the heavier tail per dimension, and the final score is the maximum of the three sums.

Strengths and limits. It needs no tuning and is cheap. It assumes anomalies sit in the tails of the marginals. Our synthetic set violates that for its clustered anomalies, which sit beside the dense cluster and so inside the body of each marginal: against that group alone ECOD's ROC-AUC is 0.126, and over all anomalies it is 0.419 ± 0.062, below chance (Section 6.4). On the 30-dimensional data, where malignant cases are extreme on several features, it is competitive with LOF (0.943). This is a limit of the assumption, not a bug, and it shows why dataset structure decides the winner [2].

4.4 Autoencoder reconstruction error

Literature. Deep anomaly detection surveys group reconstruction-based methods as one of the main categories [3][4][5]. A network trained on mostly normal data learns to compress and rebuild it; anomalies rebuild poorly. Erfani et al. combine deep feature learning with a linear one-class SVM for high-dimensional data [22].

Principle. Train an encoder f and decoder g to minimise reconstruction error, then score by it.

Strengths and limits. It scales to images and sequences and learns its own features. Its known failure is over-generalisation: a model with enough capacity reconstructs anomalies too. In our small tabular runs (225 training rows for breast cancer) it was the weakest method (0.856 ± 0.076) with large variance; it did better on the 2-D synthetic data (0.925). It is a baseline here, not a fair test of deep methods on large data.

4.5 Deep one-class and semi-supervised methods

Deep SVDD. Ruff et al. (ICML 2018) note that earlier deep anomaly detectors were trained on a different task (generative modelling or compression) and adapted afterwards. Deep SVDD instead trains a network to minimise the volume of a hypersphere enclosing the network representations of the data [13]. With centre `c` and network `phi(.; W)` the one-class objective is:

It generalises the classical SVDD (Tax and Duin, 2004) and requires design care (for example, no bias terms) to avoid collapsing every input to the centre. A PyTorch implementation by the authors is public.

Deep SAD. Ruff et al. (ICLR 2020) extend Deep SVDD to the case where a few labelled anomalies exist alongside unlabelled data: labelled normals are pulled toward `c`, labelled anomalies are pushed away, and the authors report results on par with or better than shallow, hybrid and deep competitors even with little labelled data [14]. This matches ADBench's finding that a small fraction of labels helps [2].

Not implemented here. Deep SVDD and Deep SAD need a deep learning framework, which this document's runnable code deliberately avoids. They are described from the cited papers; use the authors' repositories to run them.

4.6 Supervised classification

When labelled anomalies exist, anomaly detection becomes imbalanced classification. We include a Random Forest with `class_weight="balanced"` as an upper-bound reference (`SupervisedRF`). It had the highest average precision on both datasets and the highest ROC-AUC on breast cancer (0.990 ± 0.011, average precision 0.916 ± 0.069), showing the value of even a few hundred labels, with the standing caveat that it finds only anomaly types resembling its training labels [14].

5. Recent developments (2020 to 2026)

Deep anomaly detection has its own taxonomy. Pang, Shen, Cao and van den Hengel's review organises deep methods into three high-level and 11 fine-grained categories [4]. A 2025 survey of over 180 studies, covering 2019 to 2024, sorts deep methods into reconstruction-based, prediction-based and hybrid approaches, and highlights hybrids that combine the interpretability of traditional techniques with the flexibility of deep learning [5]. Bulusu et al. extend the problem to deep-learning systems themselves, detecting out-of-distribution and adversarial inputs at run time [23].

Weak supervision beats pure unsupervision. Deep SAD (ICLR 2020) showed that a small pool of labelled anomalies improves a one-class deep model [14]. ADBench later gave the same message at scale: with 1% labelled anomalies most semi-supervised methods beat the best unsupervised method, and no unsupervised method was statistically better than the rest [2]. The practical consequence is to collect even a few labels and to treat algorithm selection as a data-dependent decision.

Time series. Darban et al. survey deep time-series detectors with a taxonomy based on detection strategy (for example reconstruction, forecasting and distance-based) and on network type [16]. Cao et al. (IEEE Transactions on Artificial Intelligence, 2025) review generative models for the same task and recommend methods by application domain [18]. Anomaly Transformer (ICLR 2022) rests on one observation: anomalies are rare, so they struggle to form strong associations with the whole series and their attention concentrates on adjacent time points. It scores each point by the discrepancy between a learned prior association and the self-attention series association, trained with a minimax strategy, and reported state-of-the-art results on six benchmarks [17].

Foundation models and transformers. A 2025 survey examines Transformers and foundation models for visual anomaly detection and contrasts them with CNN-based methods [21]. For time series, a VLDB 2024 tutorial reviews detectors from the 1980s to the present and observes that automated model-selection methods struggle to beat a random choice, which motivates foundation models as a more general alternative [19]. The promise is tempered by an ICLR 2026 paper, *When Foundation Models are One-Liners*: for five time-series foundation-model families (MOMENT, Chronos, TimesFM, Time-MoE and TSPulse) used for reconstruction or forecasting error, anomaly detection performance did not differ significantly from simple baselines such as moving-window variance [20]. The authors conclude that the assumption "anomalies are harder to reconstruct or forecast" does not hold consistently for these models.

Benchmarks and tooling. ADBench (NeurIPS 2022) standardised evaluation across supervision levels, anomaly types and data corruption [2]. PyOD offers a single API over a wide range of detectors, from classical to neural, and now hosts ECOD [15][12]. A recurring finding in [2], [19] and [20] is that claimed gains do not always survive fair comparison with simple baselines or random choice, so careful benchmarking matters as much as new architectures.

Open directions repeated across these sources: model selection without labels [19], robustness to noisy and corrupted training data [2], interpretability [16], and finding uses of foundation models that go beyond reconstruction or forecasting error [20].

6. Code, execution instructions and results

What is implemented

Class	Method	Source
KMeansDist, GMMScore, DBSCANScore, KNNDist	Clustering and distance detectors	scikit-learn, with DBSCAN turned into a continuous score
LOFScratch	Local Outlier Factor	From scratch (NumPy)
LOFSk	Local Outlier Factor	scikit-learn, used as a check
IsolationForestScratch	Isolation Forest	From scratch (NumPy)
IFSk	Isolation Forest	scikit-learn, used as a check
ECODScratch	ECOD	From scratch (NumPy)
OCSVMScore	One-Class SVM, RBF kernel	scikit-learn
AutoEncoderScore	16-4-16 autoencoder, reconstruction error	scikit-learn MLPRegressor
SupervisedRF	Random Forest, balanced class weights	scikit-learn (labelled reference)

The three from-scratch classes are short enough to read in full. Isolation Forest builds trees with random feature and split choices and scores by path length:

The full script also contains the data generators, the evaluation loop and the plotting code. Note that the snippet for above omits an option that the full script includes.

Experimental protocol

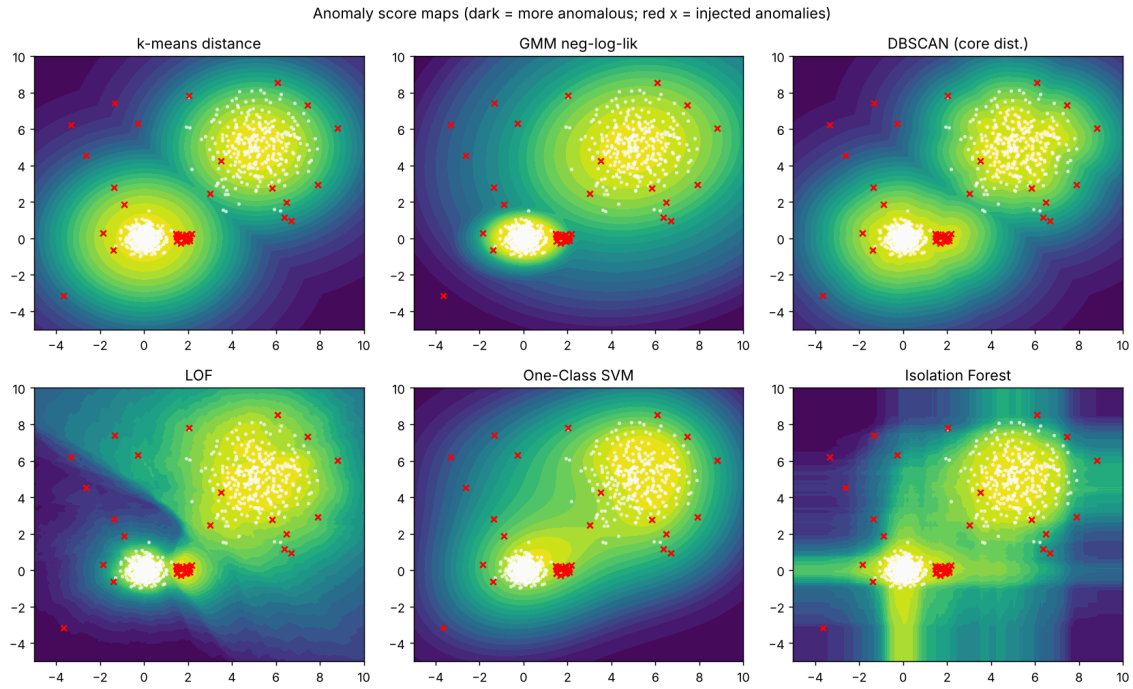
- **Synthetic 2-D data (840 points, 4.8% anomalies).** A dense Gaussian cluster (500 points), a sparse Gaussian cluster (300), 20 scattered global anomalies and 20 anomalies packed in a small group next to the dense cluster. The last group is the clustered, local kind that breaks several detectors.
- **Breast cancer, 30 dimensions (376 points, 5.1% anomalies).** The scikit-learn Wisconsin diagnostic data: all 357 benign cases are normal and a random subsample of 19 malignant cases are the anomalies.
- **Split and scaling.** Stratified 60/40 train/test split, features standardised on the training set. Unsupervised detectors are fitted on the training split *including its anomalies and without labels*, then score the held-out split. Only `SupervisedRF` sees training labels.
- **Metrics.** ROC-AUC and average precision (AP), both threshold-free. F1 is saved to `results.csv` but uses the true contamination rate as threshold, so it is optimistic and not reported here.
- **Parameters** were set by hand, not tuned: k-means and GMM use k = 2, LOF k = 20, kNN k = 10, One-Class SVM nu = 0.05, DBSCAN eps = 0.35 and `MinPts` = 8 (synthetic) or eps = 3.0 and `MinPts` = 5 (breast cancer).
- **Noise level.** The test sets hold only 16 (synthetic) and 8 (breast cancer) anomalies, so a single run is noisy. Results below are means ± standard deviation over 10 different data draws and splits.

Results

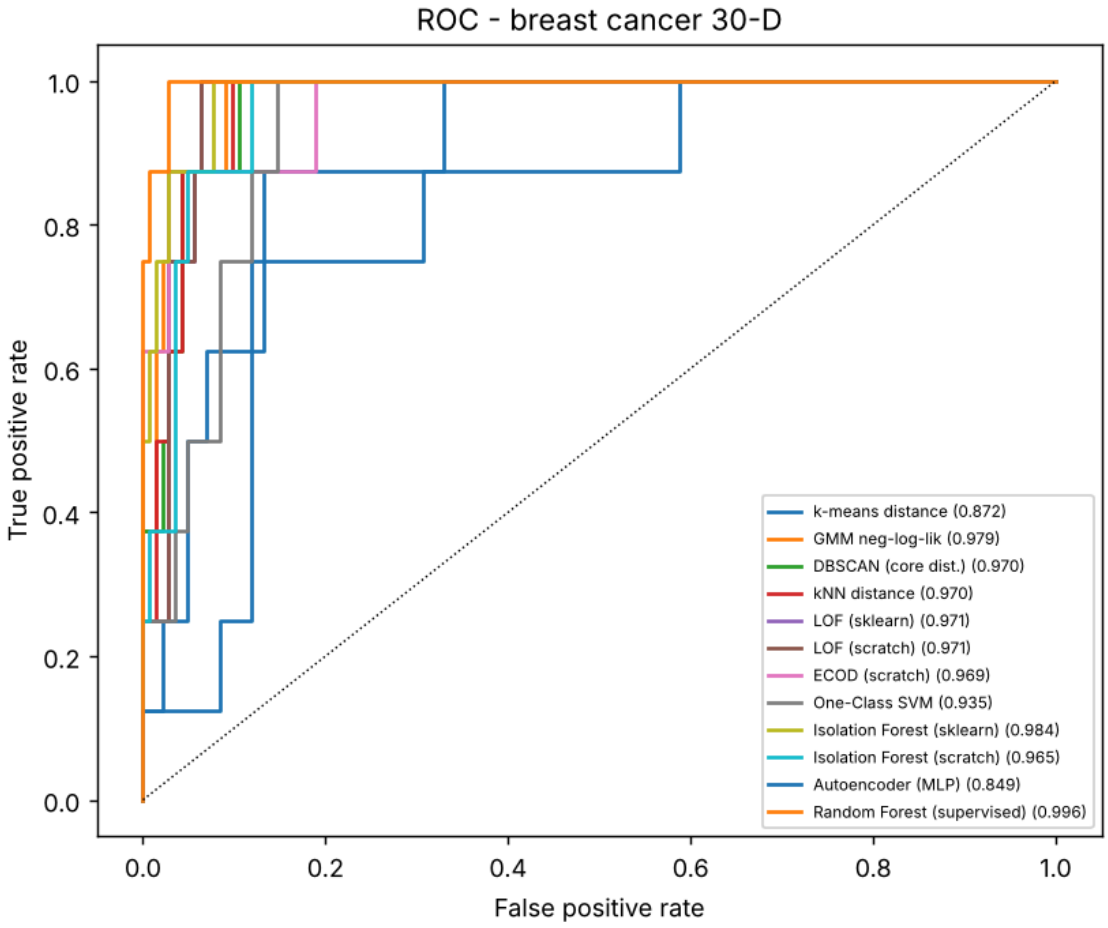
Method	**Family**	**Synthetic ROC-AUC**	**Synthetic AP**	**Breast-cancer ROC-AUC**	**Breast-cancer AP**
k-means distance	clustering	0.905 ± 0.021	0.439 ± 0.079	0.922 ± 0.047	0.548 ± 0.141
GMM neg. log-likelihood	clustering	0.965 ± 0.027	0.769 ± 0.080	0.946 ± 0.026	0.567 ± 0.163
DBSCAN (core distance)	clustering	0.767 ± 0.022	0.400 ± 0.078	0.943 ± 0.020	0.600 ± 0.126
kNN distance	distance	0.849 ± 0.041	0.424 ± 0.067	0.952 ± 0.023	0.632 ± 0.117
LOF (scratch and scikit-learn identical)	density	0.957 ± 0.035	0.715 ± 0.137	0.951 ± 0.026	0.622 ± 0.106
ECOD (scratch)	statistical	0.419 ± 0.062	0.152 ± 0.078	0.943 ± 0.025	0.634 ± 0.157
One-Class SVM	one-class	0.861 ± 0.093	0.487 ± 0.157	0.910 ± 0.032	0.448 ± 0.167
Isolation Forest (scikit-learn)	isolation	0.912 ± 0.019	0.446 ± 0.078	0.963 ± 0.018	0.664 ± 0.115

Method	Family	Synthetic ROC-AUC	Synthetic AP	Breast-cancer ROC-AUC	Breast-cancer AP
Isolation Forest (scratch)	isolation	0.893 ± 0.029	0.431 ± 0.094	0.953 ± 0.023	0.643 ± 0.122
Autoencoder (MLP)	reconstruction	0.925 ± 0.029	0.523 ± 0.153	0.856 ± 0.076	0.327 ± 0.182
Random Forest (supervised, labelled reference)	classification	0.961 ± 0.032	0.857 ± 0.090	0.990 ± 0.011	0.916 ± 0.069

- **No unsupervised method wins everywhere.** GMM leads on the synthetic data (AP 0.769), which is built from two Gaussians. Isolation Forest leads on breast cancer (ROC-AUC 0.963). This echoes ADBench's conclusion that algorithm choice depends on the data [2].
- **Unequal density hurts global methods.** On the synthetic data k-means, kNN and DBSCAN rank below LOF and GMM, which model the two densities separately. DBSCAN's single eps is the clearest case (0.767).
- **Assumptions matter more than sophistication.** ECOD, a tail-based method, scores below chance when anomalies sit in the middle of the marginals, then matches LOF on the 30-dimensional data where malignant cases are extreme.
- **Labels help.** The supervised forest has the highest AP on both datasets, consistent with the benefit of even limited supervision reported in [2]. It can find only the anomaly types it has seen.
- **Scratch implementations are faithful.** LOF matches scikit-learn exactly in ranking (Spearman 1.000). Isolation Forest matches closely (0.993 and 0.935) with a small AUC gap of about 0.01 to 0.02, which comes from differences in tree construction.
- **The autoencoder is a weak baseline here**, not evidence against deep methods: it trains on 225 rows with no tuning.



The score maps (fitted on all 840 points) show how each family sees the plane. k-means draws round bands around two centres. GMM adapts the band to each cluster's spread. DBSCAN follows the cluster shapes. LOF shows a sharp break between the two clusters and a bright patch where the small anomaly group sits, so the group partly masks itself. One-Class SVM draws a smooth boundary. Isolation Forest shows the axis-aligned banding discussed in Section 4.2.



The ROC curves are from the single seed-42 run, whose test set has only 8 anomalies, so the curves are step-like. Use the 10-seed table for comparisons.

Breakdown by anomaly type. The overall synthetic score hides a split between the two anomaly groups. The table scores each detector against the 20 scattered (global) anomalies and the 20 clustered anomalies separately, over the same 10 data draws. Detectors here are fitted and scored on all points, so values differ from the train/test table above. The function is `breakdown_by_anomaly_type` in the script.

Detector	**ROC-AUC, scattered anomalies**	**ROC-AUC, clustered anomalies**
ECOD	0.778 ± 0.053	0.126 ± 0.017
LOF	0.939 ± 0.037	0.457 ± 0.116
GMM	0.965 ± 0.018	0.975 ± 0.018
Isolation Forest	0.966 ± 0.014	0.873 ± 0.025
One-Class SVM	0.936 ± 0.031	0.891 ± 0.028

Scattered anomalies are easy for everything except ECOD. The clustered group separates the methods: ECOD scores it as more normal than normal data (0.126, well below chance), and LOF is near chance,

plausibly because the group is about the size of the neighbourhood (k = 20), so each member's neighbours are mostly other members. GMM does best because its tight covariance for the dense cluster puts the group many standard deviations away. This is a masking effect of the kind Isolation Forest's authors discuss [10]; we did not test the neighbourhood-size explanation directly.

Limitations

Two small datasets, one synthetic, cannot rank the methods in general. Deep SVDD, Deep SAD and Anomaly Transformer are not implemented. The autoencoder uses scikit-learn's `MLPRegressor`, not a deep learning framework. For larger studies use ADBench [2] or PyOD [15], which cover far more datasets and detectors.

7. Comparison, open challenges and conclusions

7.1 Choosing a method

The table turns the literature and our experiments into starting points. None is a rule: benchmark evidence says to compare several detectors on your own data [2].

Situation	**Start with**	**Why**
Tabular data, no labels, unknown structure	Isolation Forest, cross-checked with LOF or GMM	Best unsupervised on our 30-dimensional data; few parameters [9][10]
Clusters of different density	LOF or GMM	Both model local scale; k-means, kNN and DBSCAN with one global radius did worse here [6]
Anomalies may form small groups	GMM, Isolation Forest or One-Class SVM; avoid LOF with k near the group size	Section 6.4 breakdown
You need a tuning-free, explainable score	ECOD, after checking that anomalies are extreme in some feature	Parameter-free with per-dimension tail scores [12]; fails otherwise (Section 6.4)
A few labelled anomalies exist	Semi-supervised (Deep SAD) or a supervised classifier	Labels helped in [2] and [14]
Images, sequences, time series	Deep reconstruction or transformer detectors, compared with simple baselines	[17][20]

7.2 Open challenges

- **Model selection without labels.** Automated selection struggles to beat random choice for time series [19].
- **Thresholds.** Every method here returns a score; turning it into an alert needs a contamination estimate or labelled data. Our F1 numbers use the true contamination as a threshold and are therefore optimistic.

- **Contaminated training data.** Unsupervised and one-class methods assume the training set is mostly normal. ADBench tests robustness to noisy and corrupted data as one of its three angles [2].
- **Evaluation noise.** Anomalies are rare, so test sets are small. Our test sets held 8 and 16 anomalies, and standard deviations over seeds reached 0.09 in ROC-AUC for some methods.
- **Interpretability.** The time-series survey [16] devotes a discussion to model interpretability; ECOD's per-dimension scores are one simple route [12].
- **Foundation models.** Reconstruction and forecasting error from large pretrained models did not outperform one-line baselines in a recent study [20], so new ways to use them are needed.

7.3 Conclusion

Clustering and density methods ask how well a point fits its neighbours; classification-style methods learn a boundary, a path length or a reconstruction. Neither family dominates. In our experiments the ranking changed with the data and with the kind of anomaly: GMM and LOF led on a two-density synthetic set, Isolation Forest on a 30-dimensional medical set, and ECOD and LOF failed on anomalies that formed their own group. The recent literature points the same way: labels, careful evaluation and a check of each method's assumption matter more than choosing the newest architecture [2][19][20]. The accompanying script lets a reader reproduce every number and add detectors to the same harness.

8. References

Entries 1 to 25 were checked against the linked pages (as of 2026-10-27), except that the venue and year for entry 4 come from memory. Tax and Duin's 2004 SVDD paper is mentioned in Section 4.5 as cited by [13] and was not consulted directly.

1. Chandola, V., Banerjee, A., Kumar, V. (2009). Anomaly detection: A survey. ACM Computing Surveys 41(3), Article 15.
2. Han, S., Hu, X., Huang, H., Jiang, M., Zhao, Y. (2022). ADBench: Anomaly Detection Benchmark. NeurIPS 35, Datasets and Benchmarks Track. Also arXiv:2206.09426.
3. Chalapathy, R., Chawla, S. (2019). Deep learning for anomaly detection: A survey. arXiv:1901.03407.
4. Pang, G., Shen, C., Cao, L., van den Hengel, A. (2021). Deep learning for anomaly detection: A review. ACM Computing Surveys (venue and year from memory).
5. Deep learning advancements in anomaly detection: A comprehensive survey (2025). arXiv:2503.13195.
6. Breunig, M. M., Kriegel, H.-P., Ng, R. T., Sander, J. (2000). LOF: Identifying density-based local outliers. ACM SIGMOD.
7. Ester, M., Kriegel, H.-P., Sander, J., Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. KDD-96.
8. Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., Williamson, R. C. (2001). Estimating the support of a high-dimensional distribution. Neural Computation 13(7), 1443–1471.
9. Liu, F. T., Ting, K. M., Zhou, Z.-H. (2008). Isolation forest. ICDM 2008, 413–422.
10. Liu, F. T., Ting, K. M., Zhou, Z.-H. (2012). Isolation-based anomaly detection. ACM TKDD 6(1), Article 3.
11. Hariri, S., Carrasco Kind, M., Brunner, R. J. Extended Isolation Forest. arXiv:1811.02141 (2018); IEEE TKDE 33(4), 1479–1489, doi:10.1109/TKDE.2019.2947676.

12. Li, Z., Zhao, Y., Hu, X., Botta, N., Ionescu, C., Chen, G. H. (2023). ECOD: Unsupervised outlier detection using empirical cumulative distribution functions. IEEE TKDE 35(12), 12181–12193.
13. Ruff, L., Vandermeulen, R., Görnitz, N., Deecke, L., Siddiqui, S. A., Binder, A., Müller, E., Kloft, M. (2018). Deep one-class classification. ICML, PMLR 80, 4393–4402.
14. Ruff, L., Vandermeulen, R. A., Görnitz, N., Binder, A., Müller, E., Müller, K.-R., Kloft, M. (2020). Deep semi-supervised anomaly detection. ICLR 2020.
15. Zhao, Y., Nasrullah, Z., Li, Z. (2019). PyOD: A Python toolbox for scalable outlier detection. JMLR 20(96), 1–7.
16. Darban, Z. Z., Webb, G. I., Pan, S., Aggarwal, C. C., Salehi, M. (2024). Deep learning for time series anomaly detection: A survey. ACM Computing Surveys; arXiv:2211.05244.
17. Xu, J., Wu, H., Wang, J., Long, M. (2022). Anomaly Transformer: Time series anomaly detection with association discrepancy. ICLR 2022.
18. Cao, J., Miao, J., Tao, H., Wang, Y., Wu, J., Wang, Z., Wu, X. (2025). Generative models for time series anomaly detection: A survey. IEEE Transactions on Artificial Intelligence.
19. Liu, Q., Boniol, P., Palpanas, T., Paparrizos, J. (2024). Time-series anomaly detection: Overview and new trends. PVLDB 17(12), 4229–4232.
20. When foundation models are one-liners: Limitations and future directions for time series anomaly detection. ICLR 2026.
21. Foundation models and transformers for anomaly detection: A survey (2025). arXiv:2507.15905.
22. Erfani, S. M., Rajasegarar, S., Karunasekera, S., Leckie, C. (2016). High-dimensional and large-scale anomaly detection using a linear one-class SVM with deep learning. Pattern Recognition 58, 121–134 (as listed in [3]).
23. Bulusu, S., Kailkhura, B., Li, B., Varshney, P. K., Song, D. (2020). Anomalous example detection in deep learning: A survey. IEEE Access 8, 132330–132347.
24. scikit-learn documentation, sklearn.neighbors.LocalOutlierFactor.
25. scikit-learn documentation, sklearn.ensemble.IsolationForest.

Code

anomaly_detection.py

```
import time

import warnings

import numpy as np

import pandas as pd

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt

from scipy.stats import rankdata, spearmanr
```

```
from sklearn.cluster import KMeans, DBSCAN
from sklearn.datasets import load_breast_cancer, make_blobs
from sklearn.ensemble import IsolationForest, RandomForestClassifier
from  sklearn.metrics  import  roc_auc_score,  average_precision_score,  roc_curve,
f1_score
from sklearn.mixture import GaussianMixture
from sklearn.model_selection import train_test_split
from sklearn.neighbors import LocalOutlierFactor, NearestNeighbors
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.svm import OneClassSVM


warnings.filterwarnings("ignore")
SEED = 42


# Data
def make_synthetic(seed=SEED, n_dense=500, n_sparse=300, n_anom=40):
    """Two clusters of DIFFERENT density + global and local anomalies (label 1 =
anomaly)"""
    rng = np.random.RandomState(seed)
    dense = rng.normal([0, 0], 0.4, size=(n_dense, 2))
    sparse = rng.normal([5, 5], 1.2, size=(n_sparse, 2))
    glob = rng.uniform(-4, 9, size=(n_anom // 2, 2))                # scattered
global outliers
    local = rng.normal([0, 0], 0.4, size=(n_anom // 2, 2)) * 0 + \
        np.c_[rng.uniform(1.4, 2.2, n_anom // 2), rng.uniform(-0.3, 0.3, n_anom
// 2)]  # near dense cluster
    X = np.vstack([dense, sparse, glob, local])
```

```
    y = np.r_[np.zeros(n_dense + n_sparse), np.ones(n_anom)]
    return X, y.astype(int)



def make_breast_cancer(seed=SEED, anomaly_frac=0.05):
      """Standard benchmark construction: benign = normal, small subsample of malignant = anomalies"""
    d = load_breast_cancer()
     X, y = d.data, 1 - d.target              # sklearn: 1=benign -> flip so 1 = malignant
    rng = np.random.RandomState(seed)
    normal_idx = np.where(y == 0)[0]
    mal_idx = np.where(y == 1)[0]
    n_anom = int(round(anomaly_frac * len(normal_idx) / (1 - anomaly_frac)))
    keep = np.r_[normal_idx, rng.choice(mal_idx, n_anom, replace=False)]
    return X[keep], y[keep]



# From-scratch implementations
def c_factor(n):
       """Average path length of an unsuccessful BST search (normaliser in iForest)."""
    if n <= 1:
        return 0.0
    if n == 2:
        return 1.0
    return 2.0 * (np.log(n - 1) + 0.5772156649) - 2.0 * (n - 1) / n
```

```
class _ITree:
    def __init__(self, X, depth, max_depth, rng):
        self.size = len(X)
        self.depth = depth
        self.leaf = depth >= max_depth or len(X) <= 1
        if self.leaf:
            return
        spread = np.where(X.max(0) > X.min(0))[0]
        if len(spread) == 0:
            self.leaf = True
            return
         self.q = rng.choice(spread)                                    # random
feature
          self.p = rng.uniform(X[:, self.q].min(), X[:, self.q].max())  # random
split value
        mask = X[:, self.q] < self.p
        self.left = _ITree(X[mask], depth + 1, max_depth, rng)
        self.right = _ITree(X[~mask], depth + 1, max_depth, rng)

    def path_length(self, x):
        if self.leaf:
            return self.depth + c_factor(self.size)
        return (self.left if x[self.q] < self.p else self.right).path_length(x)
```

```
class IsolationForestScratch:
      """Liu, Ting & Zhou (2008). score = 2^(-E[h(x)] / c(psi)); near 1 =>
anomaly."""

    def __init__(self, n_trees=100, psi=256, seed=SEED):
        self.n_trees, self.psi, self.seed = n_trees, psi, seed

    def fit(self, X):
        rng = np.random.RandomState(self.seed)
        self.psi_ = min(self.psi, len(X))
        max_depth = int(np.ceil(np.log2(self.psi_)))
        self.trees_ = []
        for _ in range(self.n_trees):
            idx = rng.choice(len(X), self.psi_, replace=False)
            self.trees_.append(_ITree(X[idx], 0, max_depth, rng))
        return self

    def score(self, X):
            h = np.array([[t.path_length(x) for t in self.trees_] for x in
X]).mean(1)
        return 2.0 ** (-h / c_factor(self.psi_))


class LOFScratch:
     """Breunig et al. (2000). LOF ~ 1 for inliers, >> 1 for local outliers
(novelty mode)."""
```

```
    def __init__(self, k=20):
        self.k = k

    def fit(self, X):
        self.X_ = X
        self.nn_ = NearestNeighbors(n_neighbors=self.k + 1).fit(X)
        d, idx = self.nn_.kneighbors(X)
        d, idx = d[:, 1:], idx[:, 1:]                  # drop self
         self.kdist_ = d[:, -1]                              # k-distance of every
training point
        self.lrd_ = self._lrd(d, idx)
        return self

    def _lrd(self, d, idx):
          reach = np.maximum(d, self.kdist_[idx])         # reach-dist_k(p, o) =
max(k-dist(o), d(p, o))
        return 1.0 / (reach.mean(1) + 1e-12)

    def score(self, Xq, in_train=False):
        off = 1 if in_train else 0
        nn = NearestNeighbors(n_neighbors=self.k + off).fit(self.X_)
        d, idx = nn.kneighbors(Xq)
        d, idx = d[:, off:], idx[:, off:]
        lrd_q = self._lrd(d, idx)
        return (self.lrd_[idx].mean(1)) / lrd_q
```

```
class ECODScratch:
      """Li et al. (2022/2023). Parameter-free: aggregate negative-log tail probabilities per dimension."""

    def fit(self, X):
        self.Xs_ = np.sort(X, axis=0)
        self.n_ = len(X)
        self.skew_ = np.sign(((X - X.mean(0)) ** 3).mean(0))
        return self

    def score(self, X):
        n = self.n_
         left = np.stack([np.searchsorted(self.Xs_[:, j], X[:, j], side="right") for j in range(X.shape[1])], 1) / n
             right = np.stack([n - np.searchsorted(self.Xs_[:, j], X[:, j], side="left") for j in range(X.shape[1])], 1) / n
        left, right = np.clip(left, 1 / n, 1), np.clip(right, 1 / n, 1)
        o_left, o_right = -np.log(left), -np.log(right)
            o_auto = np.where(self.skew_ < 0, o_left, o_right)              # skewness-corrected tail
        return np.maximum.reduce([o_left.sum(1), o_right.sum(1), o_auto.sum(1)])



# 3. Detector wrappers: fit(train) -> score(test), higher = more anomalous
class KMeansDist:
    def __init__(self, k=2): self.k = k
         def fit(self, X): self.m = KMeans(n_clusters=self.k, n_init=10, random_state=SEED).fit(X); return self
```

```
    def score(self, X): return self.m.transform(X).min(1)




class GMMScore:

    def __init__(self, k=2): self.k = k

          def   fit(self,   X):   self.m   =   GaussianMixture(n_components=self.k,
covariance_type="full", random_state=SEED).fit(X); return self

    def score(self, X): return -self.m.score_samples(X)




class DBSCANScore:

    """DBSCAN labels noise as -1; we turn that into a continuous score = distance
to nearest core point."""

       def  __init__(self,  eps=0.5,  min_samples=10):  self.eps,  self.ms  =  eps,
min_samples

    def fit(self, X):

        self.m = DBSCAN(eps=self.eps, min_samples=self.ms).fit(X)

        core = X[self.m.core_sample_indices_]

        self.nn = NearestNeighbors(n_neighbors=1).fit(core if len(core) else X)

        return self

    def score(self, X): return self.nn.kneighbors(X)[0].ravel()




class KNNDist:

    def __init__(self, k=10): self.k = k

      def  fit(self,  X):  self.nn  =  NearestNeighbors(n_neighbors=self.k).fit(X);
return self

    def score(self, X): return self.nn.kneighbors(X)[0][:, -1]
```

```
class LOFSk:
    def __init__(self, k=20): self.k = k
        def fit(self, X): self.m = LocalOutlierFactor(n_neighbors=self.k, novelty=True).fit(X); return self
    def score(self, X): return -self.m.score_samples(X)


class OCSVMScore:
    def __init__(self, nu=0.05, gamma="scale"): self.nu, self.g = nu, gamma
      def fit(self, X): self.m = OneClassSVM(nu=self.nu, gamma=self.g).fit(X); return self
    def score(self, X): return -self.m.decision_function(X)


class IFSk:
           def fit(self, X): self.m = IsolationForest(n_estimators=200, random_state=SEED).fit(X); return self
    def score(self, X): return -self.m.score_samples(X)


class AutoEncoderScore:
    def __init__(self, hidden=(16, 4, 16)): self.h = hidden
    def fit(self, X):
            self.m = MLPRegressor(hidden_layer_sizes=self.h, activation="relu", solver="adam", max_iter=600,
                                  random_state=SEED, early_stopping=False).fit(X, X)
```

```
        return self
    def score(self, X): return ((self.m.predict(X) - X) ** 2).mean(1)



class SupervisedRF:
        """Classification framing: needs labelled anomalies (reference/upper bound)."""
       def fit(self, X, y): self.m = RandomForestClassifier(n_estimators=300, class_weight="balanced", random_state=SEED).fit(X, y); return self
    def score(self, X): return self.m.predict_proba(X)[:, 1]



class Wrap:
    """Adapter for scratch classes whose scoring method is `.score`."""
    def __init__(self, obj): self.o = obj
    def fit(self, X): self.o.fit(X); return self
    def score(self, X): return self.o.score(X)



# Evaluation
def evaluate(name, family, det, Xtr, ytr, Xte, yte, supervised=False):
    t0 = time.time()
    det.fit(Xtr, ytr) if supervised else det.fit(Xtr)
    s = det.score(Xte)
    dt = time.time() - t0
     thr = np.quantile(s, 1 - yte.mean())                    # flag the top (true contamination) fraction
```

```
    return dict(method=name, family=family,
                ROC_AUC=roc_auc_score(yte, s), AP=average_precision_score(yte, s),
                F1=f1_score(yte, (s >= thr).astype(int)), seconds=dt), s



def run_benchmark(X, y, tag, k_clusters=2, db_eps=0.5, db_ms=10, ocsvm_nu=0.05, split_seed=SEED):
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, stratify=y, random_state=split_seed)
    sc = StandardScaler().fit(Xtr)
    Xtr, Xte = sc.transform(Xtr), sc.transform(Xte)
    zoo = [
        ("k-means distance", "clustering", KMeansDist(k_clusters), False),
        ("GMM neg-log-lik", "clustering", GMMScore(k_clusters), False),
        ("DBSCAN (core dist.)", "clustering", DBSCANScore(db_eps, db_ms), False),
        ("kNN distance", "distance", KNNDist(10), False),
        ("LOF (sklearn)", "density", LOFSk(20), False),
        ("LOF (scratch)", "density", Wrap(LOFScratch(20)), False),
        ("ECOD (scratch)", "statistical", Wrap(ECODScratch()), False),
        ("One-Class SVM", "classification", OCSVMScore(ocsvm_nu), False),
        ("Isolation Forest (sklearn)", "isolation", IFSk(), False),
        ("Isolation Forest (scratch)", "isolation", Wrap(IsolationForestScratch()), False),
        ("Autoencoder (MLP)", "reconstruction", AutoEncoderScore(), False),
        ("Random Forest (supervised)", "classification", SupervisedRF(), True),
    ]
    rows, scores = [], {}
```

```
    for name, fam, det, sup in zoo:
        r, s = evaluate(name, fam, det, Xtr, ytr, Xte, yte, supervised=sup)
        r["dataset"] = tag
        rows.append(r)
        scores[name] = s
    return pd.DataFrame(rows), scores, (Xtr, Xte, ytr, yte)



def breakdown_by_anomaly_type(n_seeds=10):
    """Synthetic data only: ROC-AUC against scattered (global) vs clustered (local) anomalies separately.
    Detectors are fitted and scored on all points (transductive), so values differ from the train/test table."""
    dets = {"ECOD": lambda: Wrap(ECODScratch()), "LOF": lambda: LOFSk(20), "GMM": lambda: GMMScore(2),
            "Isolation Forest": lambda: IFSk(), "One-Class SVM": lambda: OCSVMScore(0.05)}
    out = {}
    for sd in range(n_seeds):
        X, y = make_synthetic(seed=sd)
        kind = np.zeros(len(y), int); kind[-40:-20] = 1; kind[-20:] = 2      # 1 = global, 2 = clustered
        Xs = StandardScaler().fit_transform(X)
        for name, mk in dets.items():
            s = mk().fit(Xs).score(Xs)
            for k, lab in [(1, "global"), (2, "clustered")]:
                m = (kind == 0) | (kind == k)
```

```
                out.setdefault((name, lab), []).append(roc_auc_score((kind[m] ==
k).astype(int), s[m]))
        rows = {n: {lab: f"{np.mean(out[(n, lab)]):.3f} +- {np.std(out[(n,
lab)]):.3f}" for lab in ("global", "clustered")}
            for n in dets}
    return pd.DataFrame(rows).T



# Figures
def plot_score_maps(Xraw, y, path="score_maps.png"):
    """Visualise how each family scores the 2-D plane (raw coordinates)."""
        models = [("k-means distance", KMeansDist(2)), ("GMM neg-log-lik",
GMMScore(2)),
            ("DBSCAN (core dist.)", DBSCANScore(0.5, 10)), ("LOF", LOFSk(20)),
            ("One-Class SVM", OCSVMScore(0.05)), ("Isolation Forest", IFSk())]
    gx, gy = np.meshgrid(np.linspace(-5, 10, 200), np.linspace(-5, 10, 200))
    grid = np.c_[gx.ravel(), gy.ravel()]
    fig, axes = plt.subplots(2, 3, figsize=(13, 8))
    for ax, (name, m) in zip(axes.ravel(), models):
        m.fit(Xraw)
        z = rankdata(m.score(grid)).reshape(gx.shape) / grid.shape[0]
        ax.contourf(gx, gy, z, levels=20, cmap="viridis_r")
        ax.scatter(*Xraw[y == 0].T, s=4, c="white", alpha=.6)
        ax.scatter(*Xraw[y == 1].T, s=22, c="red", marker="x")
        ax.set_title(name)
     fig.suptitle("Anomaly score maps (dark = more anomalous; red x = injected
anomalies)")
    fig.tight_layout(); fig.savefig(path, dpi=130); plt.close(fig)
```

```
def plot_roc(scores, yte, title, path):
    plt.figure(figsize=(6.5, 5.5))
    for name, s in scores.items():
        fpr, tpr, _ = roc_curve(yte, s)
        plt.plot(fpr, tpr, label=f"{name} ({roc_auc_score(yte, s):.3f})")
    plt.plot([0, 1], [0, 1], "k:", lw=.8)
    plt.xlabel("False positive rate"); plt.ylabel("True positive rate")
            plt.title(title);   plt.legend(fontsize=6.5,   loc="lower   right");
plt.tight_layout()
    plt.savefig(path, dpi=130); plt.close()



# Main
if __name__ == "__main__":
    pd.set_option("display.width", 160)
    Xs, ys = make_synthetic()
    Xb, yb = make_breast_cancer()
    print(f"Synthetic: {Xs.shape}, anomalies={ys.sum()} ({ys.mean():.1%})")
            print(f"Breast   cancer   (malignant   as   anomalies):   {Xb.shape},
anomalies={yb.sum()} ({yb.mean():.1%})")


       df_s,  sc_s,  (_,  _,  _,  yte_s)  =  run_benchmark(Xs,  ys,  "synthetic-2D",
k_clusters=2, db_eps=0.35, db_ms=8)
     df_b,  sc_b,  (_,  _,  _,  yte_b)  =  run_benchmark(Xb,  yb,  "breast-cancer-30D",
k_clusters=2, db_eps=3.0, db_ms=5, ocsvm_nu=0.05)
    res = pd.concat([df_s, df_b], ignore_index=True)
```

```
    res.to_csv("results.csv", index=False)


    for tag, df in res.groupby("dataset", sort=False):
        print(f"\n=== {tag} ===")
        print(df.drop(columns="dataset").round(3).to_string(index=False))


    # sanity: scratch implementations agree with sklearn
    for ds, sc in [("synthetic", sc_s), ("breast-cancer", sc_b)]:
          r1 = spearmanr(sc["Isolation Forest (scratch)"], sc["Isolation Forest
(sklearn)"])[0]
        r2 = spearmanr(sc["LOF (scratch)"], sc["LOF (sklearn)"])[0]
                print(f"[{ds}]  Spearman  scratch-vs-sklearn:  iForest={r1:.3f}
LOF={r2:.3f}")


    # ---- robustness: repeat with 10 different data draws / splits, report mean
+- std ----
    if "--quick" not in __import__("sys").argv:
        reps = []
        for sd in range(10):
            Xs_, ys_ = make_synthetic(seed=sd)
            Xb_, yb_ = make_breast_cancer(seed=sd)
                reps.append(run_benchmark(Xs_, ys_, "synthetic-2D", 2, 0.35, 8,
split_seed=sd)[0])
             reps.append(run_benchmark(Xb_, yb_, "breast-cancer-30D", 2, 3.0, 5,
0.05, split_seed=sd)[0])
        allr = pd.concat(reps)
          g = allr.groupby(["dataset", "method"], sort=False)[["ROC_AUC", "AP",
"F1"]].agg(["mean", "std"]).round(3)
```

```
        g.to_csv("results_10seeds.csv")
        print("\n=== mean +- std over 10 seeds ===")
        print(g.to_string())


    bd = breakdown_by_anomaly_type()
    bd.to_csv("results_by_anomaly_type.csv")
    print("\n=== synthetic: ROC-AUC by anomaly type (mean +- std over 10 seeds) ===")
    print(bd.to_string())


    plot_score_maps(Xs, ys)
    plot_roc(sc_s, yte_s, "ROC - synthetic 2-D", "roc_curves.png")
                plot_roc(sc_b,    yte_b,    "ROC    -    breast    cancer    30-D", "roc_curves_breast_cancer.png")
              print("\nSaved    results.csv,    score_maps.png,    roc_curves.png, roc_curves_breast_cancer.png")
```

Result

https://drive.google.com/drive/folders/1PwAuGMSezTxwX0XsYe0a4Jqjyk4MkRFZ?usp=sharing